ECE/COE 1896

Senior Design

Origami Robot Conceptual Design

Team 8

Prepared By: Jacob Ferrer
William Roper
Zachary McPherson
Karthik Raja

Table of Contents

Table of Figures

Table of Tables

1. Introduction

The Origami Robot is a self-folding robotic system that can transform itself from a flat sheet into different 3D shapes, allowing it to move through tight or constrained spaces where rigid robots cannot operate. The mechanical structure of the robot will be comprised of individual tiles that are interconnected by hinges. By folding these hinges, the robot will be able to morph into different 3D shapes to meet the needs of its environment. This kind of robot can be used in various applications; however, this design will focus on the inspection and exploration of an indoor environment. Equipped with cameras and sensors, the robot will be able to enter a room and generate a real-time 3D reconstruction of the space while maneuvering around various obstacles. The Origami Robot will represent a hybrid of rigid and soft robotics that will be capable of navigating unknown environments with the potential for use in other applications.

This project aims to tackle the difficult problem of producing a low-profile, yet capable robot for operating in complex environments. Prior work has resulted in robots with impressive capabilities to either survey an environment or navigate through tight spaces, but rarely both. However, many applications require a robot that does both; applications include, but are not limited to, maneuvering around debris and surveying pipes/ductwork. Despite maintaining a small form, the robot should make no compromises on its ability to collect and transmit useful information about the environment.

The goal of this project is to design and build a fully functional prototype that shows both the folding mechanism and 3D reconstruction of an environment. The robot will autonomously maneuver in a room, adapt to the environment by changing its shape, and use its on-board sensors and cameras to produce a real-time mapping of the environment. The system will consist of electrical, software, and mechanical subsystems to demonstrate its shapeshifting and spatial awareness capabilities.

The prototype will serve as a proof of concept for two use cases that require access to tight and confined spaces while also maintaining spatial awareness. The first use case is search and rescue, where the Origami Robot would help first responders survey potentially hazardous environments before entering. The other use case is inspection, where the Origami Robot would traverse difficult to reach areas, such as pipes or HVAC ducts, and then detect any abnormalities in these areas.

The system block diagram for the Origami Robot, found below, provides a high-level overview of the major subsystems and their interactions with other modules around the four main roles of the project: Control Lead, Power Lead, Firmware Lead, Software Lead. The Control Lead manages the robot's overall behavior and coordinates hinge actuation to guide its movement. The Power Lead manages the distribution of stable and reliable power to the various components of the robot. The Firmware Lead manages low-level operations, such as sensor polling, distributing control signals, and communication between the robot and high-level software. The Software Lead manages high-level computation, including 3D reconstruction, SLAM, folding instructions, and user control.

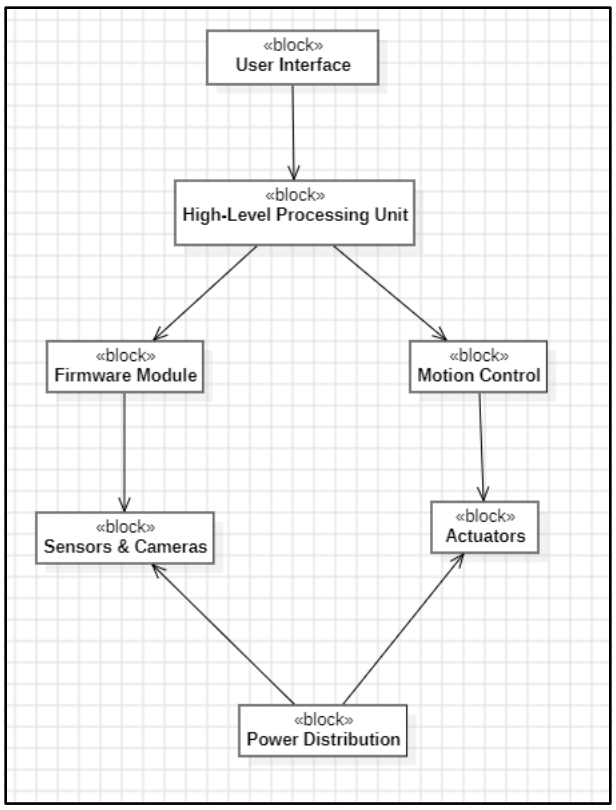

Figure 1: System Block Diagram

2. Background

Traditional robotics are designed with specific conditions in mind; however, increased complexity and variability of the environment degrade robot performance. This limitation becomes critical when first responders require surveillance data to assess emergency situations but are prevented due to unknown environmental challenges such as locked doors, rubble, and tight spaces. In such environments a hybrid of soft and rigid robotics is necessary.

2.1 The Use Case

Our first targeted hurdle for the Origami Robot to handle is being able to access rooms behind doors. During emergencies, a key door may be locked, and first responders need to know the room or environment structure to develop an emergency procedure. The 1958 fire of Our Lady of the Angels School is a historical example of this scenario. In this tragedy, 92 students died due to the fire in the building. Part of the reason so much damage was done was because the emergency door was locked and first responders were unaware of the full situation. A historical source described the events of the tragedy as the “Evacuation of the second floor of the annex and south wing was hampered by smoke which came through an open door in the division wall at the second story level. This door was closed early in the fire, possibly by a fireman” [1]. In other words, the students were trapped in the building, and the first responders were unable to assess the situation because of the locked door. Therefore, it was lack of knowledge that led to an inability to access the situation. Through a hybrid of soft and rigid robotics the Origami Robot would be able to

maneuver under the critical door and alert the first responders that there were children trapped inside and where the first responders could access the floor to save the trapped people.

The second problem we are going to address with our project is regarding collapsed buildings caused by earthquakes. These occurrences will result in rubble and debris across the environment, which will be inaccessible for human surveillance. Small gaps and cracks prevent from first responders from finding collapse survivors and saving lives. Researchers from Germany used an example of a building collapse in Cologne, Germany to stress the need of safety robotics. Their description of the scenario states: "On March 3rd, 2009, the historical archive of the city of Cologne (Germany) and two adjacent residential buildings collapsed into a building excavation of the local subway. Initially, it was unclear how many victims were affected by this event. Thus, the Fire Department of Cologne initiated a major alarm." Due to the uncertainty of the situation, the department enlisted the help of survey robots. These robots failed to work due to the incompatibility of their size with the environment necessary to traverse. Therefore, this example highlights a specific scenario where humans were unable to safely search through the area and needed a robot to go through the area more efficiently. The prototype of our project will be addressing these problems and making our robot so that it can be used in future scenarios explained above.

2.2 The Solution

Our proposed solution is a modular robot that can fold itself into a variety of structures while using computer vision to make a 3D reconstruction of its surrounding area. This robot ecosystem will solve miniature problems, fit a multitude of use cases and scale for mass production.

As the industrial world begins to scale down its commercial solutions, safety solutions need to be scaled too. Rescue robots have become a popular idea in natural disaster scenarios, such as in earthquakes or fires. In an article reporting on the surveillance robot "FirstLook" [2], the author states that "Smaller rescue robots can also fit into tight spaces that humans can't — or shouldn't — navigate." In other words, the smaller the robot the more terrain that it can traverse safely. Of course, our origami would not do the "saving" part of the rescue mission, but it would be able to map the ground that it has covered, allowing for humans to safely enter the area knowing the obstacles that are ahead. The robot wouldn't be constrained to situations where safety is jeopardized; its small size would allow it to fit in pipes for surveillance purposes. Maintaining a small and modular robot will allow for further expansion and the ability to explore environments that are inaccessible to humans.

Safety solutions are tied to use cases: the more problems a robot can solve, the more effective a product it is. An example being, if a mobile robot can traverse over obstacles but not under the obstacle, then it will get stuck at the obstacle and no longer be a viable solution. In their analysis of structural collapses and their subsequent robotic solutions, researchers out of Carnegie Mellon University concluded that "The complexity of the rubble with the different types of collapse modes may favor a modular robotics approach in order to adapt to the different types of confined space environments in different collapse areas" [3]. The researchers also stated that in order for the solution to be affective it must be adaptable. The use of the word "different" multiple times stresses for safety solutions to have modular robotics. Our solution will cover this two-fold: the robot being able to fold depending on its terrain and be scalable to add additional "origami tiles" for different applications.

Safety solutions also need to be economically viable, both in terms of cost and in terms of manufacturability. Berkeley researchers designing crawling robots for similar safety purposes expounded that "[Miniature robots with] low weight and cost allow their deployment in large numbers, independently or in swarms, to cover a large work area and increase the odds that some of the robots will succeed in performing a specific task" [4]. The researchers acknowledged that perfect performance may not exist; however, a modular and expandable solution will circumvent this problem. Our Origami Robot shall be a cost-effective design so that first responders can afford to use this robot in bulk to speed up their rescue efforts.

Safety and surveillance solutions must be compact, adaptable and cost effective to allow for widespread use. Our Origami Robot will solve these constraints through being modular and foldable.

3. System Requirements

This section outlines the several different system requirements for our system. These requirements include functional requirements, performance requirements, and additional requirements as well. Functional requirements are specified actions that a system must perform to satisfy user or stakeholder needs. On the other hand, performance requirements explain how well a system shall perform the functional requirements. They are not essential to the system being complete but are nice-to-haves. Additional requirements often are related to security, scalability, and maintainability to make sure that the system is properly updated and operated over time.

3.1 Functional Requirements

3.1.1 Geometry – Vertical Height

The robot shall be no more than 1 inch tall when in a flat position to navigate through narrow openings.

3.1.2 Control – Hinge Actuation

The Origami Robot shall have a tiled frame capable of folding into different shapes. The hinge actuation shall be robust enough to move the weight of the robot to help it navigate obstacles.

3.1.3 Control – Expandability

The hinge mechanism, tile construction, and movement system shall be repeatable across many tiles.

3.1.4 Control – Robot Movement

Movement of the robot shall allow it to remain in a flat, compact form factor. It should be done precisely in a control loop, which integrated position data and object detection.

3.1.5 Power – USB Battery Charging

The power system shall receive USB input to charge 3.7V LiPo battery.

3.1.6 Power – Battery Monitoring

The power system shall be able to monitor the charge level of the 3.7V LiPo battery. Turns off if voltage drops below 3V.

3.1.7 Power – Rail System

The power system shall run on a 3.3V sensing rail (500mA) and a motor rail (1A).

3.1.8 Power – Disconnect System

The power system will be able to turn off modules (sensing, motor drives, motor hinges) depending on the microcontroller.

3.1.9 Firmware – Sensor Integration

The firmware shall interface with all of the sensors embedded on the robot, such as the IR sensor, IMU sensor, and the Camera, in order to acquire data at the sampling rate.

3.1.10 Firmware – Wireless Communication

The firmware shall transmit data from the sensors and camera wirelessly from the first ESP 32 to the second ESP 32 using the communication protocol, ESP NOW.

3.1.11 Firmware – Wired Communication

The firmware shall forward data from the second ESP 32 to the Orange Pi via a wired connection, such as USB.

3.1.12 Firmware – Data Packaging

The firmware shall compress and package the raw data obtained from the sensors and camera into well-defined data packets in order to make the decoding process easy for the Orange Pi.

3.1.13 Firmware – Error Handling

The firmware shall re-attempt any transmission after detecting any transmission errors, packet loss, or corrupted data.

3.1.14 Firmware – Real-Time Data Acquisition

The firmware shall collect and transmit data continuously from the sensors and camera throughout the network of microcontrollers to support real-time data collection.

3.1.15 Software – Sensor Integration

The software shall receive and interpret sensor data such as stereo images, IMU readings, and IR sensors.

3.1.16 Software – Navigation

The software shall use a stereo SLAM algorithm to estimate its own pose, where to go next, and detect repeated locations.

3.1.17 Software – 3D Reconstruction

The software shall use stereo images to create a 3D visualization of an indoor space.

3.1.18 Software – Manual Control

The software shall be able to relinquish control of the robot's movement to the user.

3.2 Performance Requirements

3.2.1 Control – Hinge ROM

The hinge mechanism shall allow the moving tile to have a range of motion of at least 90° in either direction from the plane of the adjacent tile.

3.2.2 Control – Motor Control Loop

The response of the motor control loop shall have sufficient parameters, such as rise time, settle time, and steady state error to achieve reliable and efficient functionality of the robot.

3.2.3 Power – Battery Operating Range

The battery shall operate from 3.0V - 4.20V. The power system will be cut off if battery voltage is lower than 3.0V.

3.2.4 Power – 3.3V Logic Rail

The 3.3V rail shall have a continuous current of 500 mA and allow for current spikes under 900 mA without damaging hardware.

3.2.5 Power – 5V Motor Rail

The 5V rail shall operate at a continuous current of 1A and allow for current spikes of 1.5A. These current spikes shall only happen under stall current conditions.

3.2.6 Firmware – Latency

The firmware shall make sure that the data transmission from the first ESP to the Orange Pi occurs within 50 milliseconds.

3.2.7 Firmware – Throughput

The firmware shall provide an efficient data rate in order to successfully transmit all sensor and camera data without any loss.

3.2.8 Firmware – Reliability

The firmware shall have the ability to achieve a successful packet transmission rate of 95% throughout the operation of the Origami Robot.

3.2.9 Firmware – Bit Error Rate

The firmware shall have the ability to achieve a bit error rate of, such as 10^{-6} throughout the operation of the Origami Robot.

3.2.10 Software – Algorithm Optimization

The all algorithms shall be able to run, in real-time, on a regular sized SBC such as an Orange Pi or others like it.

3.3 Other Requirements

3.3.1 Firmware – Scalability

The firmware shall be implemented in such a way that adding more sensors or communication modules shall result in a seamless and easy integration.

3.3.2 Firmware – Maintainability

The firmware shall be well-documented in order to keep a version history regarding bugs and other updates.

4. Design Constraints: Standards and Impacts

This section outlines the several different design constraints for our system. These design constraints explain the limitations that has a chance of affecting the design and implementation of our system. Unlike system requirements, design constraints are not based on the customer's needs, but are based on other factors, such as time, budget, manpower, the learning curve, and the physical size limitation as well. These constraints help us focus on what we can physically accomplish throughout the design and implementation process.

4.1 Design Constraints

4.1.1 Time

This project must be completed within one semester's time, which includes designing the robot, implementing the system, testing all components, and preparing documentation for it as well. This time constraint of around 12 weeks limits the amount of complexity we can incorporate into this project. It also requires us to manage our tasks efficiently and break the project down into manageable modules that each of us can work on. Working on this project will allow us to practice time management in order to avoid any bottlenecks during any stage of the implementation process, which allows us to meet the system requirements mentioned above.

4.1.2 Budget

This project will be heavily constrained by the 200-dollar budget we have been given to complete it. This budget will affect the components and number of components we buy as well as the design on certain parts of the robot we are implementing ourselves to make sure that we do not cross the given budget. We shall design circuits to be as cost-efficient as possible.

4.1.3 Manpower

The Origami Robot shall be designed and implemented by our team of four students, who each have a unique set of skills in software, hardware, and firmware. This constraint affects how roles are assigned, where each team member focuses on a unique module without creating bottlenecks or dependency on other modules to slow the project down. Since each team member oversees their own module, their ability to assist with tasks outside of their assigned module may be limited.

4.1.4 Size

The physical dimension of the robot is one of the biggest constraints our team has for this project. Our goal is for the robot to be able to go underneath average door, which have a ½ - ¾ inch gap to allow for the robot to go through. This means that we have to integrate the PCB, sensors, cameras, and wheels within the robot and still have a height between ½ - ¾ inch. This may be a stretch for the scope of this project, due to necessary components exceeding this height, so we will aim to achieve a height of less than 1 inch. Through careful planning of placements for the components, we can make sure that all the needed components are included, while still being able to debug and access the robot for adjustments in the future.

4.1.5 Learning Curve

All the members in our team must learn new concepts, such as efficient power distribution, motor control methods, Orange Pi/ESP 32 interfacing, and 3D reconstruction algorithms as well. We must learn, design, and implement this within the project's timeline. The steepness of the curve limits how much we can expand our project, but we are designing the robot in a way that allows for easy expansion in the future.

4.1.6 Standards

All components that are involved in this project must agree with any relevant safety or communication standards, such as voltage limits, USB/serial communication protocols, and wireless transmission regulations. Making sure that our project sticks to these standards make sures that the project will not be a safety hazard, has no hardware damage, and works in such a way that it can be brought to the real world.

4.2 Impacts on Non-Technical Contexts

This section outlines the several different non-technical impacts our system might have. These non-technical impacts explain the different impacts our design might have on the environment, public health, society, cultures, economy, safety, and diversity as well. These impacts help us to make a design that takes all of these into consideration and have as much of a positive impact as we can.

4.2.1 Environmental Impact

The design for our robot includes incorporating electronic components, such as ESP 32 chips, cameras, sensors, and Orange Pi, which all need power. The project will also include materials such as batteries and PCBs as well, which has the potential to contribute to e-waste if not dealt with properly. To minimize negative impacts, we aim to use low-power components to reduce energy consumption and ensure that materials can be reused or recycled where possible, which is also helpful for sustainability.

4.2.2 Public Health Impact

The robot itself poses very little risk to the public health of people, but the design should still make sure that the robot is safe to operate indoors. For example, if there is a person inside of the room we are currently reconstructing, we should make sure that sharp edges, unstable mechanical movement, or overheating is avoided to protect users.

4.2.3 Global, Cultural, and Societal Impact

The robot being developed is going to be able to autonomously map/reconstruct environments across many societal domains, such as save-and-rescue missions, room inspection, and monitoring as well. The fact that this robot would be able to do this reduces risks for humans by allowing robots to explore dangerous or inaccessible areas first. But privacy might be an issue with the cameras since it can accidentally capture sensitive data. Globally, as mentioned above, this robot would be able to support relief efforts and enhance safety inspections. Regarding the cultural impacts, the robot could even reconstruct historical sites for research purposes or just for others to view.

4.2.4 Diversity, Equity, and Inclusion Impact

The design of the robot will consider inclusion by enabling people with unique technical backgrounds to have access to this product. This will be done by clearly documenting our design and any changes we decide to make. Using low-cost electrical components also allows more people in the world to have access to this, since the cost of this design would be relatively low compared to other robots. Another way we support accessibility is by making sure that the UI is intuitive and does not contain unnecessary complexity.

4.2.5 Welfare and Safety Impact

Safety is one of the most important concerns for our system. Safety risks, such as electrical risks, unwanted mechanical movement, and sharp surfaces are risks that we plan on focusing on for our design. Having a manual override feature, size limitations, and proper power distribution help prevent such risks from happening. Regarding the welfare of the public, this robot can improve well-being by assisting in environments that may otherwise pose hazards to humans, such as collapsed structures or confined spaces. However, this technology could be used improperly. Surveillance and military applications are other potential uses of the robot described in this document that we would not advocate for. To avoid the event of this robot being used for nefarious purposes, blinking lights or speakers may be integrated.

4.2.6 Economic Impact

Affordable and expandable robots that help map environments, such as the robot we are making, help reduce the costs for industries like construction and disaster reliefs. By designing the project within our 200-dollar budget, it lowers the need for expensive industrial equipment and can stimulate innovation at all levels of the economic hierarchy. For example, in construction, faster and more accurate mapping can reduce project delays and labor costs. In disaster recovery, autonomous mapping robots can speed up search-and-rescue operations. The development of such robots could help start growth in the robotics field, creating new jobs and strengthening the U.S. economy with the new product and jobs made.

5. Conceptual Design

This section will present the conceptual design for our Origami Robot system. One of the main goals of this section is to think of multiple ways our team can approach the design for the robot, while still coming to the same end goal. Each of the modules for our robot will have two different ways the corresponding lead can go about implementing it. By evaluating the trade-offs between each of the design proposals, we intend to take the design that is most efficient and matches most with our constraints and explain the reasoning for the specific design we decided to go with for each module.

5.1 Design Concepts

5.1.1 Controls Module

The Controls Module will feature designs for the actuation of the hinges between tiles and the movement of the overall robot. This will include the hardware for motor drivers, the mechanical interactions to achieve the desired motion, and the software control schemes.

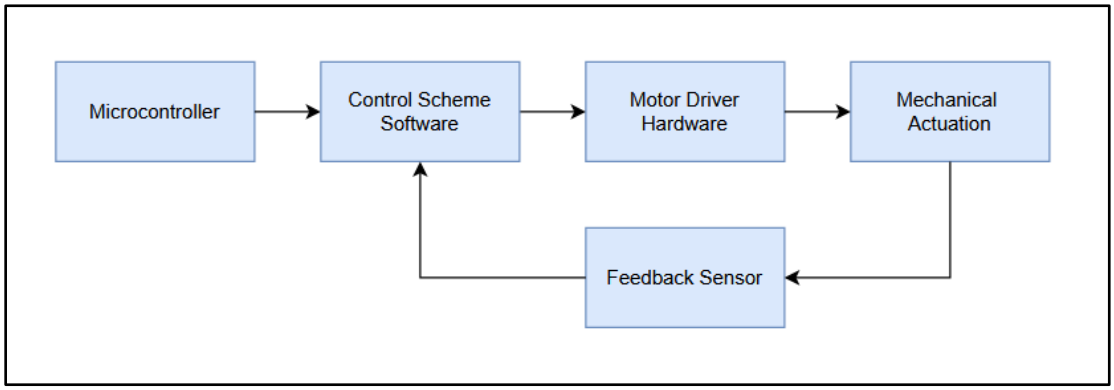

Figure 2: Overall Controls Module Diagram

5.1.1.1 Design Concept 1

Hardware Design

The first iteration of the hardware for the motor drivers is based on the DRV8833 chip by TI. This chip is widely used in dual H-bridge driver implementations, making it ideal for small dc motors. It relies on MOSFETs rather than BJTs, offering better heat dissipation than older drivers. Breakout boards for this chip are widely available and inexpensive, making it a sensible option for the tight budget of this project, even in the testing stage.

A schematic for a DRV8833 breakout board is included below. Besides the chip itself, only a few passive components are necessary, so this design could be easily duplicated and implemented in a large custom PCB for the final prototype. Note that the driver boards would likely not be used in the final robot. Instead, the ICs with other components would be integrated into the main PCB where needed.

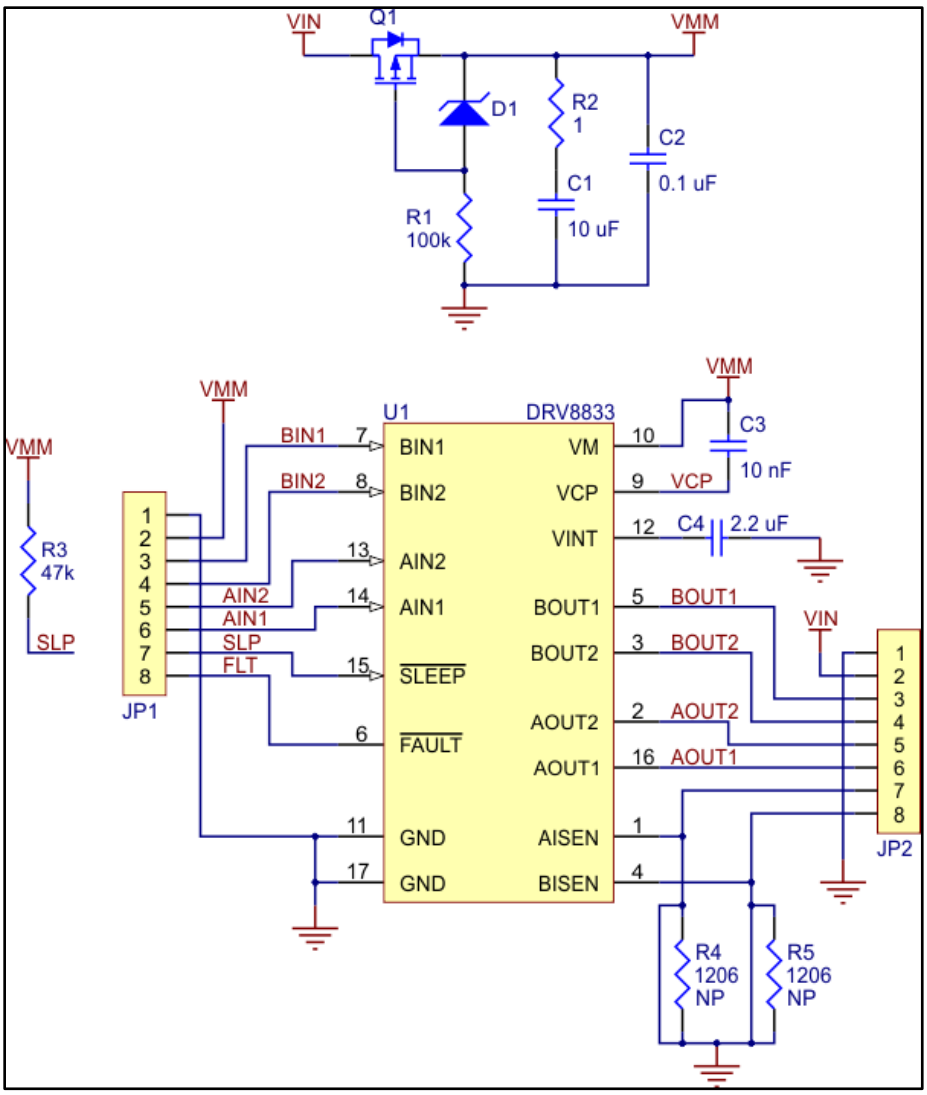

Figure 3: Schematic of a DRV8833-Based Motor Driver

The integration of this motor driver setup with the motors themselves and the microcontroller is straightforward. Motors can be directly connected to the driver's outputs, and two inputs for each motor can be connected to the microcontroller. From these connections, the motors can be turned on, turned off, and driven at variable speeds via a PWM signal. For each IC, two N20 motors can be driven at variable speeds in either direction. As per the TI data sheet, local bulk capacitors must be implemented between power and ground of the driver supply to avoid voltage drops from motor current demands [18].

Additional benefits of this design include fault detection, overcurrent and overvoltage protection, and sleep mode. This would ensure redundancy in the protection scheme of the overall circuit, which is always beneficial in system architecture. Also, these features are already included in the DRV8833 chip, only requiring minimal passive components for an application circuit. This is an advantage over the second conceptual design, which is much more basic in its features and requires extensive use of discrete components, rather than an IC.

Mechanical Design

For the robot's desired functionality, hinges and a method of translational movement must be developed. This design will feature N20 metal gear motors for both the hinges and wheels, which will add to the simplicity of the design without sacrificing any functionality. The overall footprint of the robot will be 3 square tiles of equal size. Two actuating hinges will link the tiles together.

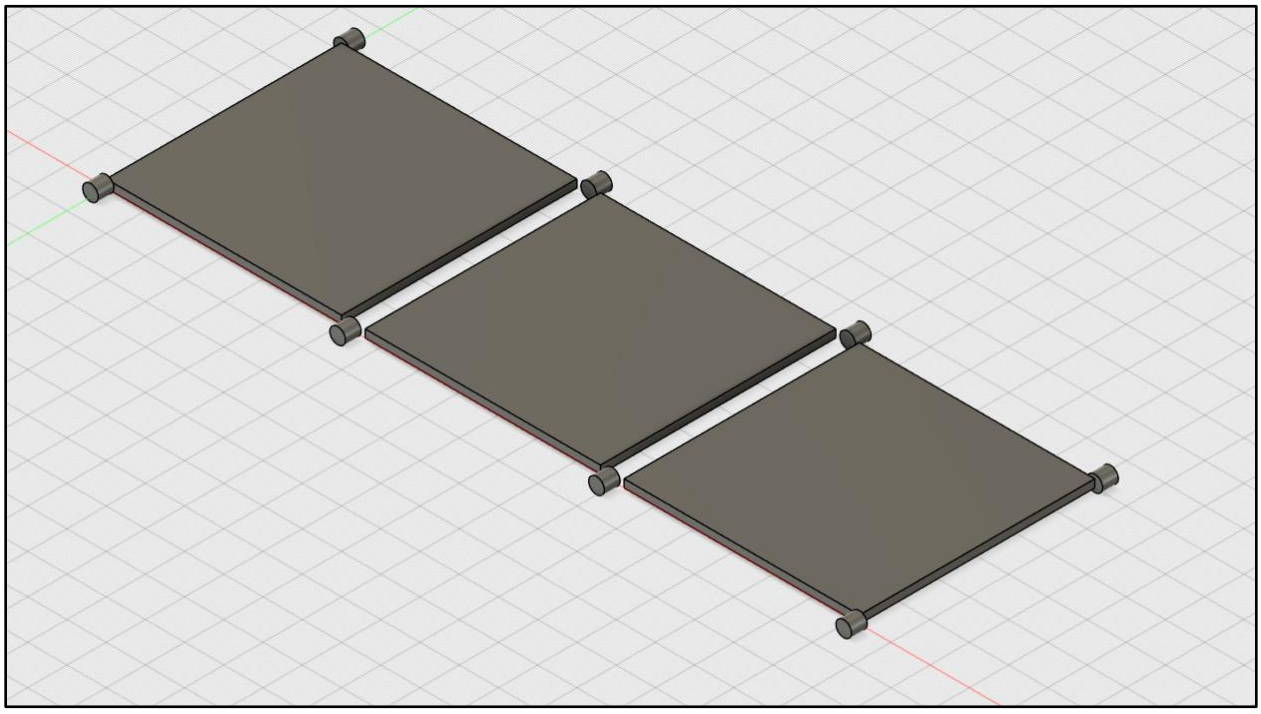

Figure 4: Overall Robot Footprint

The hinge mechanism will be a combination of Lego and 3-D printed parts which will mount to the PCB tiles. A Lego axle will be fixed to one of the tiles, and another Lego axle will be mounted to a motor fixed to the adjacent tile. A 3-D printed part will keep the axles captive, allowing the gears to mesh and the axles to rotate. Thus, when the motor drives its axle, the gears will mesh and turn the other axle. Since this second axle is connected to the other tile, that tile will rotate about the axis of its connected axle. A potentiometer or encoder will be used as feedback for the angle of the actuating tile relative to the adjacent tile. Assuming no slippage of the gears, which is a safe assumption, the rotation of the axle can be converted to the angle between actuating tiles.

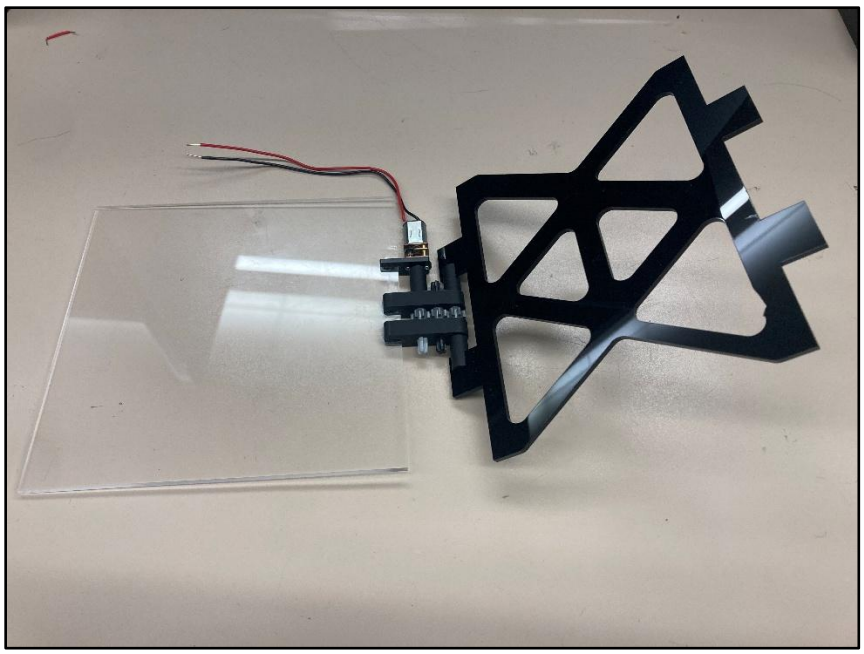

Figure 5: Acrylic Hinge Prototype

To provide the robot with the capability of translational motion, wheels will be mounted to each tile edge. These wheels will be directly mounted to N20 DC motors, which already have an integrated gearbox. These are the same motors used for the hinges, which will reduce complexity within the system. The wheels may be 3-D printed with O-rings for traction, or a premade wheel may be modified and used.

Overall, this design attempts to reduce mechanical complexity as much as possible while still retaining the core functionality of the robot. N20 geared motors ensure plenty of torque for direct drive applications but also have a very small form factor, being only 10mm tall.

Control Loop Implementation

PID (proportional, integral, and derivative) control is widely used and is a simple but effective control scheme. It features 3 contributing coefficients to scale the controlled variable toward a setpoint. In this case, PID control would be used to control the motorized hinges. An angle setpoint could be provided in software, and the PID control loop would drive the motor to achieve this angle based on the feedback of a potentiometer or encoder. Parameters such as rise time, settle time, overshoot, and steady state error can be optimized with PID as well.

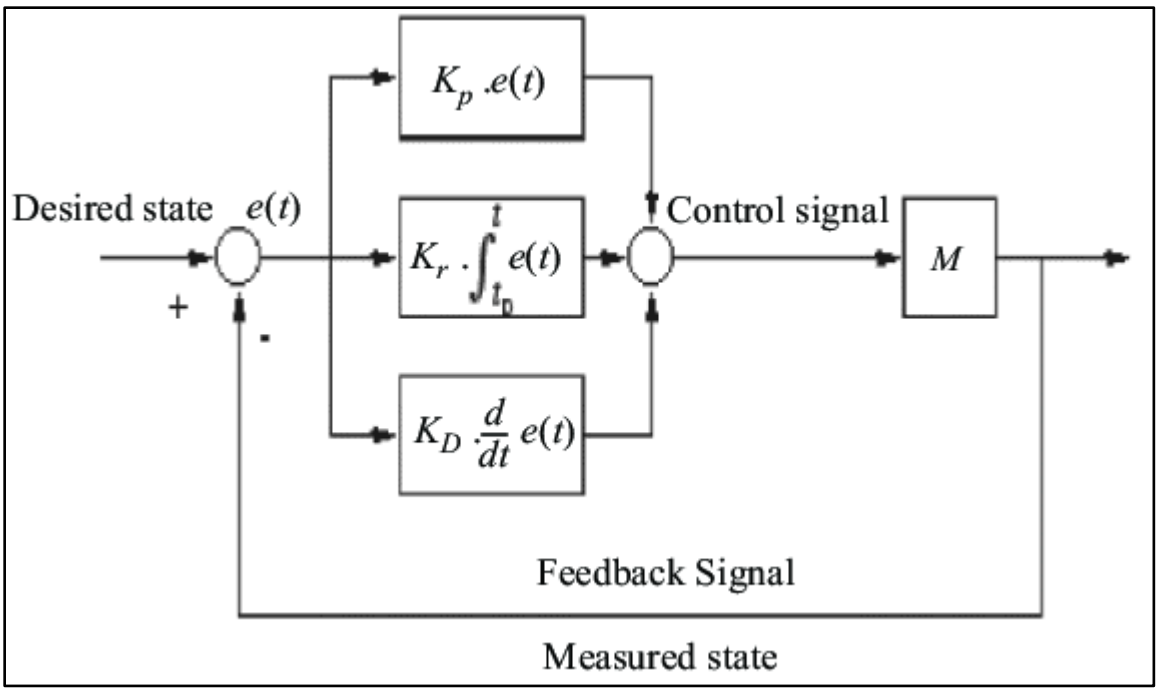

Figure 6: Hinge Control Loop

While the theory of PID control is fairly clean and straightforward, implementing it in software requires some additional calculation and considerations. The first step in creating usable control equations is to convert from the continuous frequency domain to the discrete frequency domain. From there, difference equations can be derived, which will be easily implemented in software.

The chosen method of this process is the Tustin transform, where s is substituted for $\frac{2}{T}\frac{z-1}{z+1}$. Once the control equations are in the discrete frequency domain, terms can be rearranged to produce difference equations for the proportional, integral, and derivative contributions to the controller output that can be implemented in software.

$$p[n] = K_P \cdot e[n]$$
$$i[n] = \frac{K_i T}{2}(e[n] + e[n-1]) + i[n-1]$$
$$d[n] = \frac{2K_D}{2\tau + T}(e[n] - e[n-1]) + \frac{2\tau - T}{2\tau + T} d[n-1]$$

The $p[n]$, $i[n]$, and $d[n]$ terms represent the respective PID contributions from each branch of the control loop for the nth iteration. These contributions sum up to produce the next controller output, $u[n]$,of the system.

$$u[n] = p[n] + i[n] + d[n]$$

Before implementing these equations in code, several additional considerations must be made, which can be found below. Note that the implementation of all these considerations may be redundant, but it is a valuable learning opportunity to implement them regardless.

Derivative Term amplifies high-frequency noise

In the frequency domain, the derivative branch is K_D*s, which has a frequency response that is increasing to infinity (zero at origin). This amplifies any high-frequency noise in the system, which is not what we want. To remedy this, a low-pass filter can be implemented in the form: $\frac{1}{s\tau+1}$.

Derivative Term produces "kick" when setpoint shifts

When the setpoint shifts, the error signal experiences a discontinuity, which leads to the derivative being effectively infinite. This can produce unwanted results in a real control system. Derivative on measurement can be used instead to avoid this derivative spike. Instead of calculating $d[n]$ using the error derivative, it can be calculated using the measured process variable: encoder angle feedback, in our case.

Integrator Windup

If the setpoint is far away from the current location of the process variable, the integral contribution can continue to accumulate, which may cause undesirable overshoot or instability. Clamping the integral contribution with specific thresholds can remedy this.

Controller Output Limits

Like the integral contribution was clamped, it is good practice to limit the overall controller output with specific thresholds. Note that limits of voltage and amperage will be set when configuring the stepper motor to protect the system hardware separately. The limits discussed here are simply for the PID output.

Sample Time

The frequency with which the feedback sensor is sampled is critical in the control loop. The controller, by rule of thumb, should have at least 10 times the bandwidth of the plant.

$$T_C < \frac{T_{sys}}{10}$$

Example code for a PID control scheme with these practical considerations implemented can be found online from the Phil's Lab YouTube channel. [19]

Implementing the considerations above in the hinge control loop will allow us to take advantage of the robust theory of PID control while maintaining safeguards to prevent damage to the physical system.

5.1.1.2 Design Concept 2

Hardware Design

The second option for motor drivers is a dual h-bridge design based on discrete MOSFETS. This design has fewer features than the TI chip described above but can be built with more widely available components.

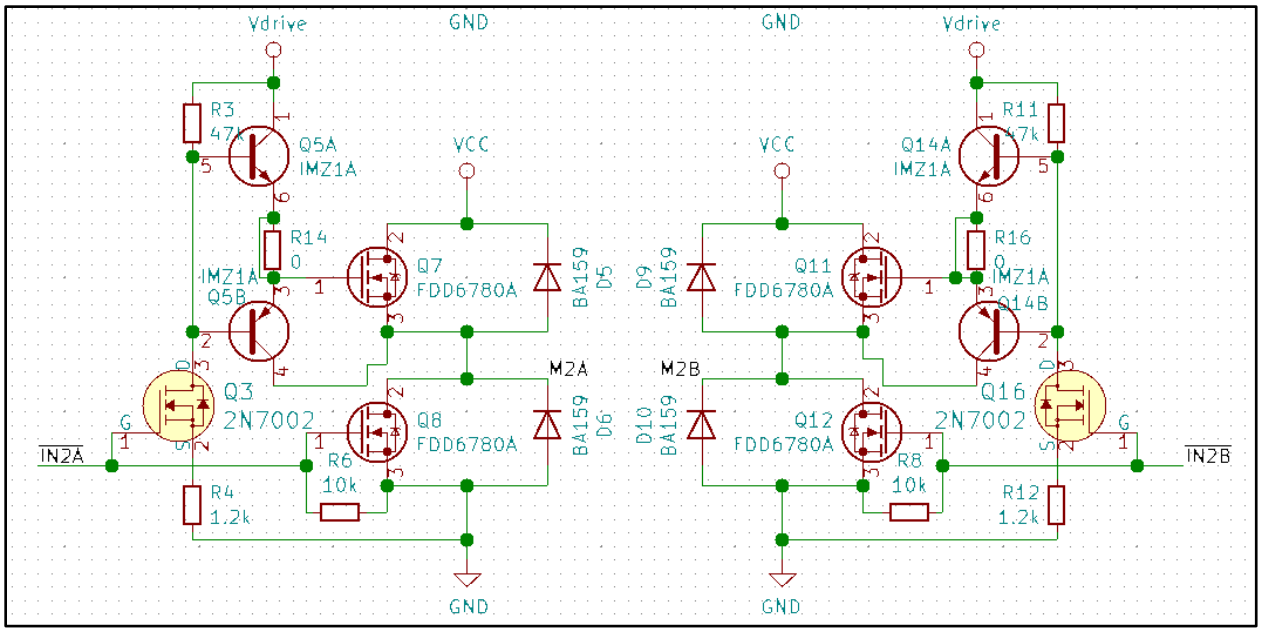

Figure 7: Secondary motor driver schematic based on discrete components

This dual H-bridge design is very bare-bones, but still retains the necessary bi-directional functionality, which is required by both the hinge mechanism and wheels. The software integration with this driver will remain largely the same as the first design, with a PWM signal being used to determine speed and direction.

One benefit to this design is that it is completely flexible in terms of current and voltage ratings of the components used. Depending on the robot's current requirements, more robust components may be necessary, which could be sourced without changing the schematic design above. The same may not be possible with an IC, where current and voltage thresholds must be observed for a given chip.

This design also has flexibility in terms of space on a PCB. Using discrete components will allow for optimizing space around other components or modules. However, PCB space is not much of a concern for this project, since the robot is being optimized for the lowest height, but not necessarily area.

Mechanical Design

For the sake of this project being focused on ECE applications, no secondary mechanical layout will be outlined for the tiles. While proceeding with the first design, any modifications to the layout will be made to reduce complexity or achieve functionality that is not present in the current design.

However, a secondary hinge design, supported by SMA wires, will be detailed here. Nitinol SMA (Shape Memory Alloy) wires have the capability of returning to an initial shape when heated. This is a convenient characteristic when integrated with electronics, as current flowing through a wire heats it up. This means that the SMA wires could have an unpowered and powered shape, allowing actuation between these states by adding or removing a voltage across them.

For the purposes of an actuating hinge, the SMA wires would be soldered between the PCB tiles to secure them and act as a conductive path from which to power them. The best way to power these wires uniformly is with a PWM signal. Conveniently, a PWM signal is also used to power the N20 motors described in the first hinge design, so a similar H-bridge circuit would be applicable to this design. However, the wires could likely be arranged in parallel and powered by the same signal to cut down on the number of drivers required.

One potential drawback to the nitinol wires is that they seem to have binary behavior in their state transition. This means that it may be difficult to control the position of the wire between their powered and unpowered states. This is a significant limitation, as the accurate actuation of the tiles throughout their range of motion is a key functionality requirement of this project. Another unknown is the actuating strength of the wires. If the hinges are used to pull the robot over an obstacle, that would present a significant load on the hinge mechanism. The nitinol wires may not be as strong as the geared motors in this respect.

The benefits of the nitinol wires lie in their small profile and simplicity. This approach greatly reduces the number of parts in the hinge mechanism and easily meets the height constraint of the robot.

Control Loop Implementation

The previous control loop design concept covered PID control for the hinges. This section will aim to extend that discussion to account for controlling the position of the robot itself and direct the driving of the wheels.

Due to budget constraints, open-loop N20 geared motors will be used for each of the 8 wheels. To provide feedback for the acceleration, an onboard IMU will record acceleration data. This data can be used for acceleration feedback, or integrated to estimate the velocity of the robot, which could also be the control variable.

Based on online research, some suggest that an IMU alone is not enough to provide reliable spatial control for a robot. To mitigate compounding errors, IR sensors will be located around the perimeter of the robot to prevent it from running into obstacles. Also, two cameras mounted to the robot will be taking in data for scene reconstruction. This information could also be integrated into the control loop, though this would increase complexity of the system.

5.1.2 Power Module

5.1.2.1 Design Concept 1

Power Tree Concept

The block diagram below summarizes the basic power system.

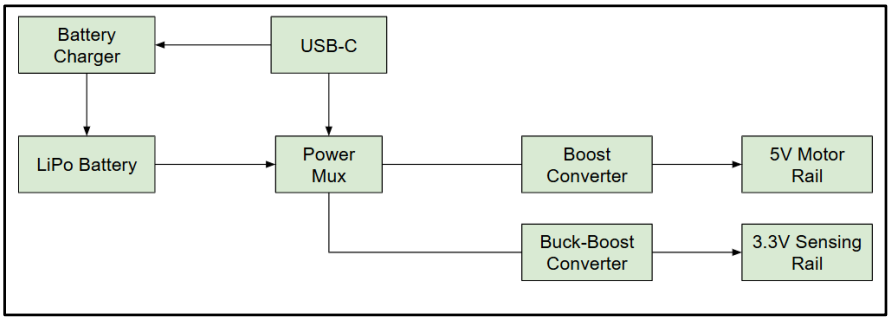

Figure 8: Power Block Diagram

The system works as follows: the USB-C simultaneously charges the LiPo battery through the battery charger and powers the system through the power mux. When the user disconnects the USB-C, the LiPo battery powers the system. The system has two rails, a motor rail and a sensing rail, whose voltage is set by a boost converter and a buck-boost converter, respectively.

Lithium-Polymer Battery

The power system is designed around one micro-Lithium polymer battery. Lithium batteries have multiple advantages over alternatives, such as NiMH batteries, but the main advantage for this project is its high-power density and light weight [5]. One drawback to the lithium battery is its safety hazard. An academic study done on lithium-ion batteries state that "Lithium-ion batteries… …[cause] frequent fires and explosions limit their further and more widespread applications." The listed drawbacks are extreme and unlikely, but still possible and therefore the design be created with these hazards in mind [6].

Therefore, to prevent safety hazards only battery packs with protection circuitry already integrated will be used. Protection circuitry will not only prevent reverse voltage accidents but also provide over-charge and over-discharge protection as well as being in "ship-mode" when not being used. "Ship-mode" is simply a condition where the battery will not discharge to the load while being unused. This feature will prevent excess voltage loss while the robot is being stored or shipped.

Charging Circuitry

The input of the charging system uses a 5V USB-C charging port. This port is popular and conveniently small for our Origami Robot. This port will then be connected to the charging IC. This IC needs to take in 5V from the input port and step down to 3.3V at 1A to charge the battery. Additionally, the IC needs to stop transmitting current once the battery is fully charged as well as notify the user when the battery is charged [7].

The IP2312, a single-cell lithium battery synchronous switch buck charging IC, is used in the Origami Robot's design. Popular alternatives to this IC include the TP4056 and the BQ25185, but both options are linear options and have high thermal losses. However, the IP2312 has a switching design reducing the chips thermal losses and running at higher efficiency. This characteristic of the chip will minimize power losses as well as reduce charging time. Reducing charging time will increase how frequently the robot can be operated, which is important to our robot because in emergency situations operators use the robot frequently and often. Below is the schematic on the IP2312's respective data sheet [8].

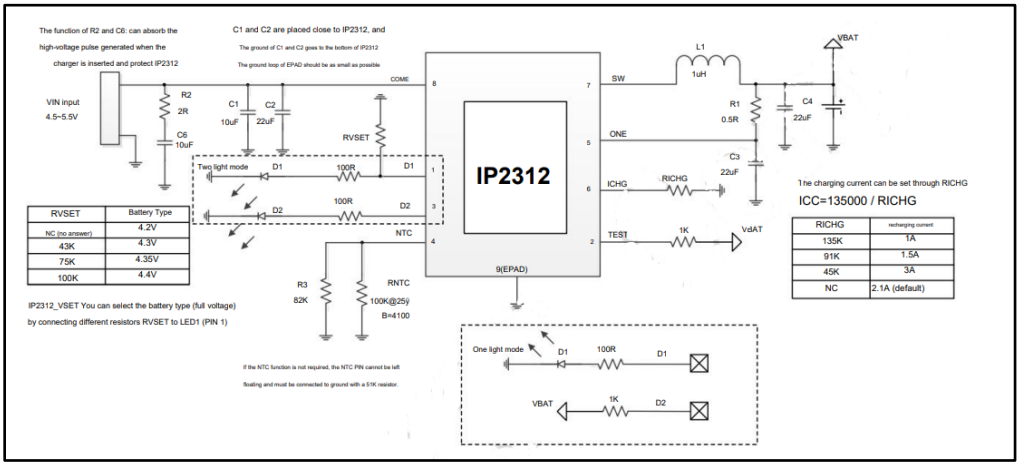

Figure 9: IP2312 Datasheet

A concern that may arise from the use of this chip is the lack of battery protection in the IP2312. This is not a problem as LiPo battery packs have built in over-current and thermal protection [9].

The components listed on the sample schematic are surface mount components. The layout will follow the rules listed in the datasheet such as the capacitors C1 and C2 being placed as close as possible to the IC ground. Lastly, the IC provides pins to add LEDs to monitor when the device is being powered and when the device is finished charging.

The TPS2116, a 1.6 V to 5.5 V, 2.5-A Low IQ Power Mux with Manual and Priority Switchover is chosen for this application [10, 11]. This IC is added to manage the direction of power while the system is charging or in other words it is a power management multiplexer. This chip allows the robot to use the USB power when it is charging, meaning sensor and ESP data can transfer without charging speeds slowing down. Additionally, including the TPS2116 allows the microcontroller to be updated and tested while the battery is being charged. This feature will benefit the engineers working on the device and save crucial time waiting for the battery's charge.

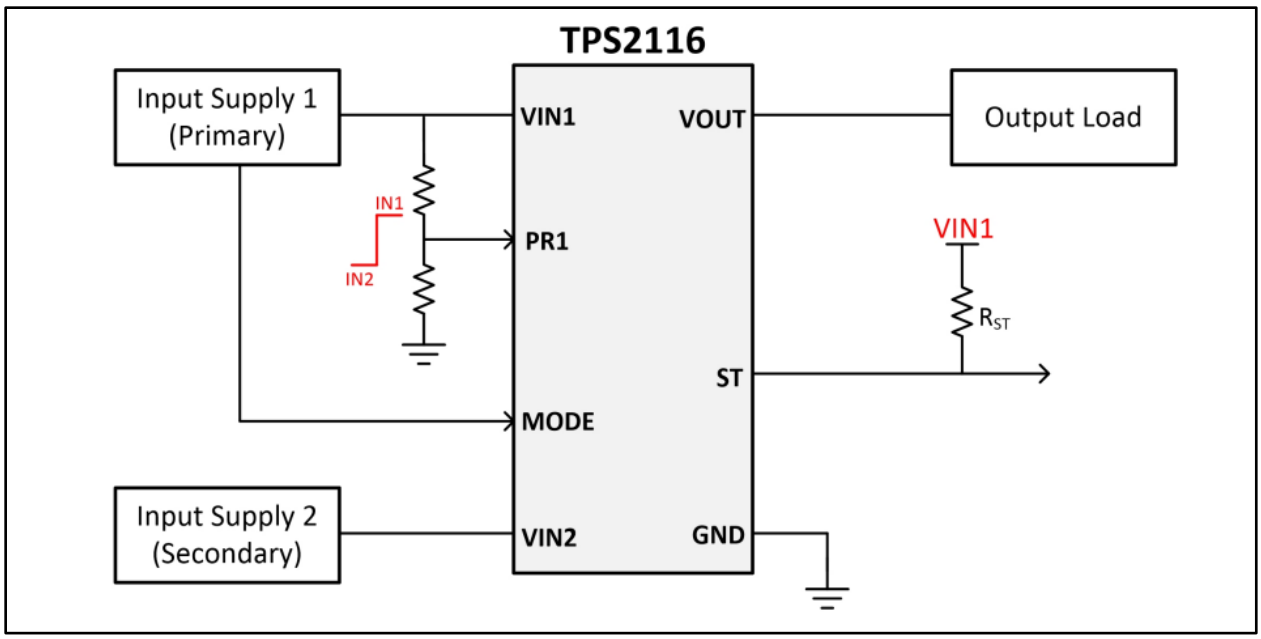

Figure 10: TPS2116 Datasheet

The setup for the TPS2116 is straightforward. VIN1 is tied to the USB input, VIN2 is tied to the positive terminal of the battery. The issue with this setup is during the time the USB-C is connected, VOUT becomes 5V, but when the USB-C is disconnected and the battery is supplying power to the MUX, VOUT becomes ~3.7V. This change in voltage is problematic to the sensor rail and the motor rail because their operating conditions will be different depending on the active voltage source. Thankfully, the buck-boost and boost converters will circumvent this issue (the converters will be discussed later). As with the previously discussed IC the surrounding circuitry will be surface mounted components whose values are to be determined.

Protection Schemes

An important part of the power circuitry is protection of sensitive devices such as the IMU and microcontroller. A practical robot must always function, and faulty, noisy currents cannot decay performance. The first device to improve protection of the power rails is a voltage protection device. This device monitors the voltage of the sensing rail and determines if the voltage has fallen below the desired voltage. This functionality prevents the microcontroller and sensors from attempting to operate while the battery supplies under regulated voltage. This problem occurs if the motors pull more current than the system is designed to manage. The TLC77-EP will suffice for this application. Its schematic is shown below in which protection is tied to a microcontroller through the ~RESET pin.

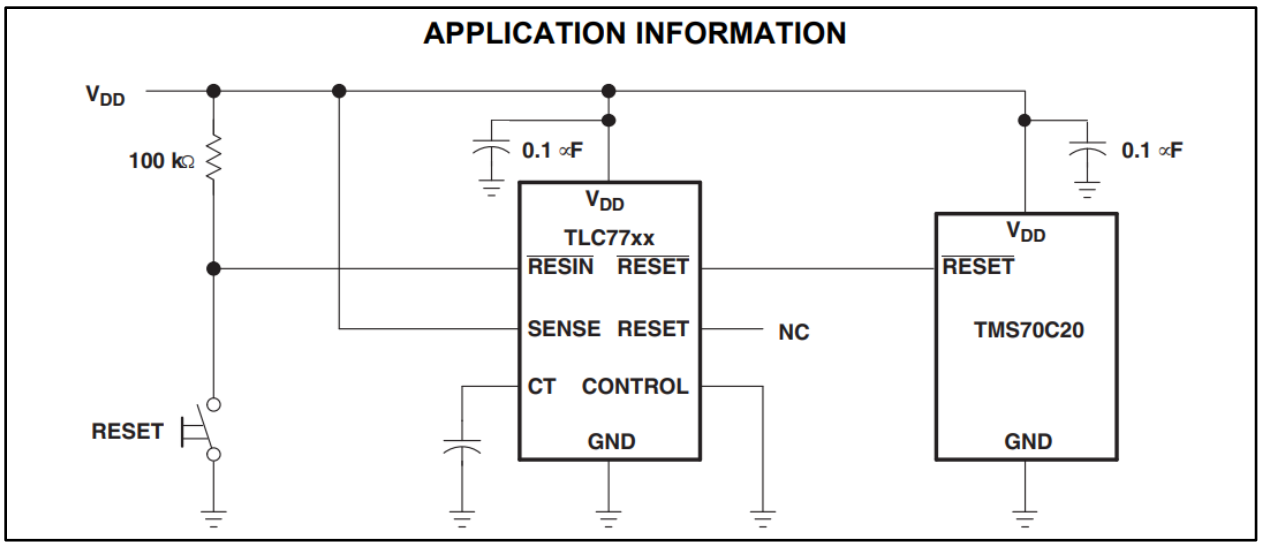

Figure 11: TLC77-EP Datasheet

For conceptual design purposes, the reset pin will only connect to the microcontroller, however as the design progresses the reset pin may need to be tied to the camera module or IR sensors to protect them as well. The TLC77xx will only monitor the sensing rail because the motor rail is assumed to be working under all conditions. Additionally, the motors do not handle sensitive data as the microcontroller and sensors do, so protecting against minor voltage peaks and dips is excessive. All additional components for the module will be surface-mounted and follow the specifications of the datasheet.

Protection schemes must protect the voltage source from inrush and transient currents. For this reason, a fuse is placed close to the battery. The fuse will be fast acting to prevent large transient currents from entering the battery acting as a emergency measure. The fuse is rated after the loads are tested to choose the appropriate fuse based on current draw. Carefully choosing the fuse is important, as resetting the fuse is expensive for the final user, and replacing a battery is equally as expensive.

Other protection mechanisms may be added later, however the devices listed above form the necessary protection needed to protect the Origami Robot's components.

Power Rails and Power Converters

As mentioned previously, the power tree system will consist of two rails to manage current loads: a sensing rail and a motor rail. These rails will prevent interfering transients as well as conserving power by separating voltage levels to match the level the devices are designed for. The sensing rail will be run on 3.3V as the specified voltage input of the ESP32 and sensors is 3.3V. Using 3.3V voltage level over 5V also reduces unnecessary power loss over the PCB traces. For our conceptual design, the motor rail will run on 5V. This rail is set higher as the motors torque and accuracy benefits form higher voltage. However, this voltage will change if during testing it is found that the motors run more efficiently on either a higher or lower voltage. The motor efficiency will be tested simply by sweeping the voltage across the motors while measuring the current draw.

The sensing rails voltage will be set by a commercial buck-boost converter. We choose a buck-boost for several reasons. The first reason is during the time the battery is fully charged at operating at 3.7V to 4.2V the voltage will need to be stepped down to 3.3V, requiring buck action. Then after long use the battery voltage will operate between 3.3V and 3V, requiring the voltage to be stepped up and therefore requiring boost action. Finally, when the robot is being charged the power mux will set Vin to 5V, requiring buck action again. Testing will be done on the sensors connected to this rail to determine the true current and voltage levels, then the buck-boost will be selected based on these specifications. Waiting to choose the device after testing is done allows the efficiency of the buck-boost to be as close to 100% as possible. Of course, no solution is perfect but to reiterate power saved anywhere leads to longer effective runtime of the Origami Robot.

The buck-boost converter the TPS63030 is shown below. The converter model will change when load testing has been accomplished, and it is determined that there are more effective buck-boost converters for the sensing rail.

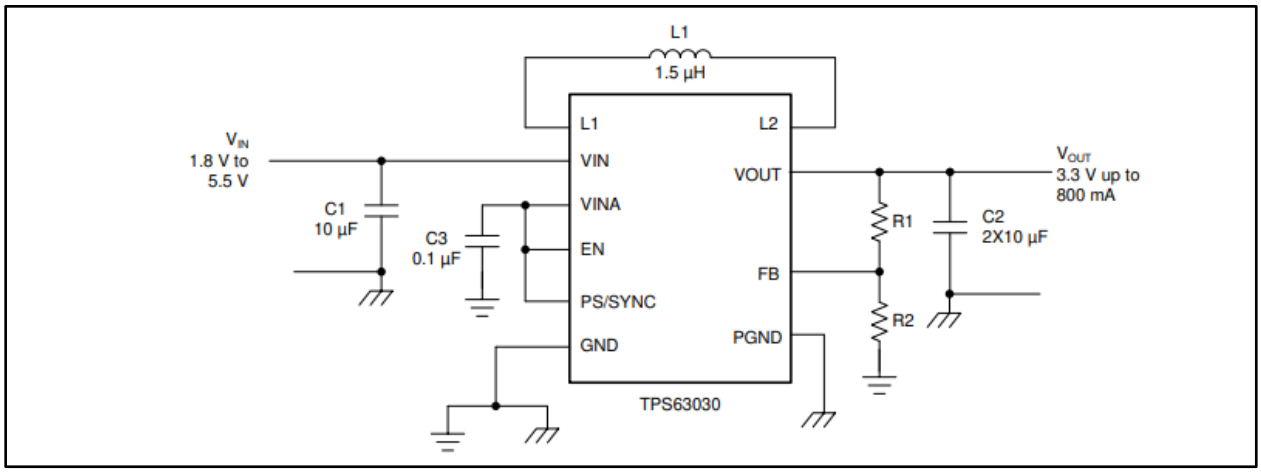

Figure 12: Example Buck-Boost Convertor Datasheet

Discrete components are surface mounted. Note that for this device capacitors and inductors must be placed as close to the IC as possible to minimize ripple currents.

The motor rail's voltage will be set by a commercial boost converter. In comparison to the sensing rails, the motor rails are less dependent on a constant voltage. Some would argue that a boost converter is unnecessary for this design and the motors can run straight of the battery voltage. While this is certainly true, for our design the motors must be precisely moved in accordance with the control loop of the robot, and keeping the voltage unmonitored would add additional complexity to the control loop that is not desired. Below is an example schematic from the TPS61253A.

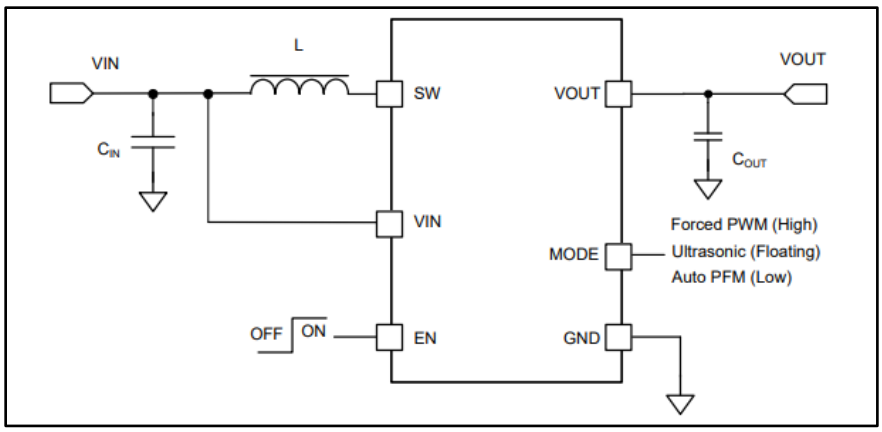

Figure 13: Example Boost Convertor Datasheet

The design methodology for the boost converter is the same as the buck-boost converter described above. To note, when the USB-C is connected and supplying 5V to the system, the boost converter will not function efficiently. However, this is acceptable as the motors will not be used when the device is charging. So only quiescent current draw will be converted by the boost converter ineffectively. To clarify the enable pin will be tied to a capacitor as in the buck-boost diagram.

Main Tile PCB Design

The main tiles of the Origami Robot will be made from a PCB. This choice entails that placement of components is crucial to the size constraints of the robot. Additionally, the tiles that rotate will put the most strain on the system, for this reason the bulk of weight, the components, will be placed on the center tile. Not only will the center tile require the bulk of the components, but it will also house 6 motors. With these constraints in mind the conceptual design is laid out below, with both a top view and a cutout view of the PCB layers.

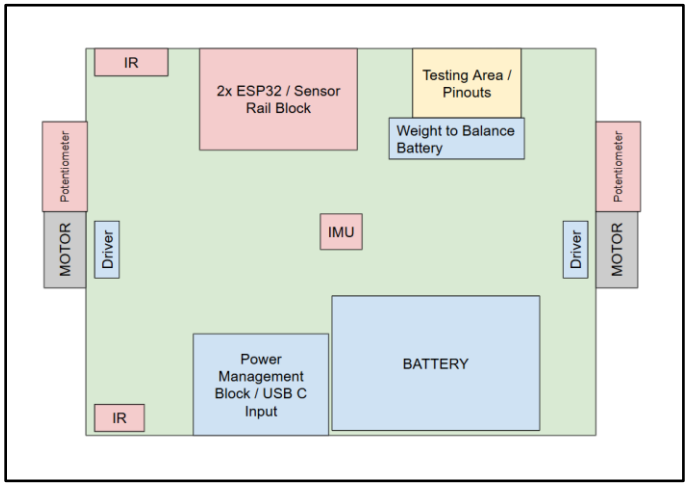

Figure 14: Base Tile Top View

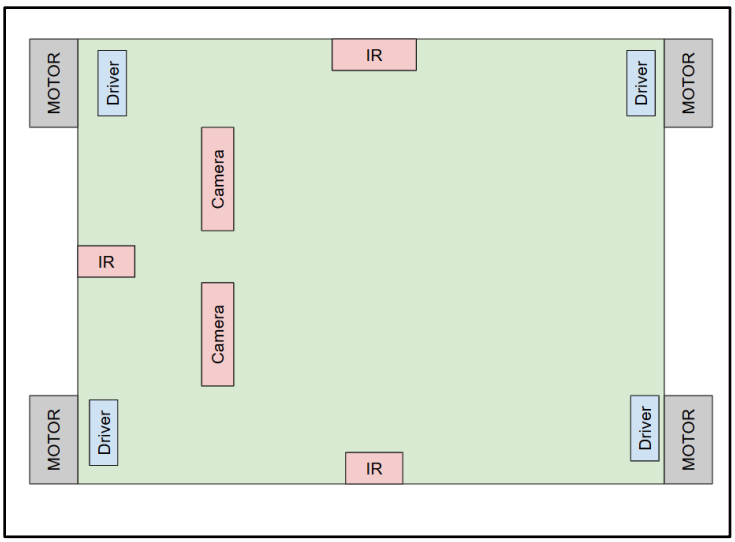

Figure 15: Side Tile Top View

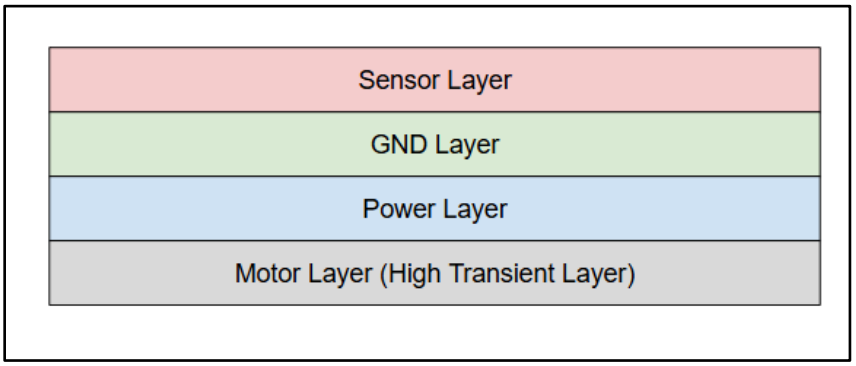

Figure 16: PCB Cross Section View

There are multiple design decisions going on in this layout, but for the sake of conceptual brevity, only the key choices will be discussed in this document. The first and most important design choice in this PCB layout is separating each block into its own section. The power management block and the sensor block will be kept separate. The motor drivers also have their own special space along the edge of the PCB. This will reduce the electromagnetic inference from one device to another. It is important to note that each motor has a respective driver that will be laid throughout the across the side and main PCB tiles. This layout of the motors leads to a circular current path that experiences frequent switching currents. TI's recommend practice states, "This line loop creates an antenna that can radiate electromagnetic energy which is determined by the current amplitude, the repetition frequency of the signal, and the geometrical area of the current loops. It is recommended to minimize these current loops for optimal EMC performance" [12]. For this reason, the bottom layer of the PCB will be dedicated to the routing of the motor signals and motor power. Additionally, this layer of the PCB will be split into an analog and digital to minimize interference.

The other important design feature of this layout is how our board is layered. Our design follows the layout given by analog devices in their guidelines on mixed-signal board layout [13]. The sensor layer will be at top to prevent any signal interference from the power layer and the motor layer. It will also be directly next to the GND layer to allow for quick return current loops. The GND layer and the power layer will be next to each other to allow for natural coupling capacitance. As mentioned earlier the motor layer will be the at the bottom to keep it most separate from the other signals.

Flexible PCB

Flexible PCB forms the connection between the side tiles and the main tile. Mark recommended this solution to our group, and this solution will be chosen instead of using jumper wires or connecting the boards signals through electromagnetism. This design choice was made because flexible PCB are more durable than jumper wires and will take up less space then both alternatives. The flexible PCB transmits motor currents and camera current, but these signals will be divided into two layers to avoid interference between the signal types. The flexible PCB is separately designed from the rigid PCB to avoid steep manufacturing costs [14]. It is soldered onto the board to form the physical connections between the materials.

Side Tile Design

The side tiles have the same layer layout as the main PCB to reduce manufacturing costs. The front tile will have 4 motors, 2 cameras, and 3 IR sensors. Motors rails will be routed away from the camera and IR sensors traces to avoid EMI and ground current spikes. The back PCB will have 4 motors and 3 IR sensors and will follow the same routing as the front tile. The weight of PCB will be measured when the first design is completed to see if the force of gravity is to great for the hinge motors. If the load is too strong for the motors, cutouts will be strategically placed in the PCB to reduce the load on the motors.

5.1.2.2 Design Concept 2

Design Concept 1 outlines the basics of a battery power system including components that are required for the robot's system and cannot be swapped or augmented. Thus, this section will only cover alternatives. It can be assumed that if a power module is not mentioned in design concept 2 it will be the same as design concept 1.

Number of Batteries

The first alternative to the system power design is the number of batteries that the system uses. The first design only used one 2000mAH 3.7V cell, an alternative would be using two or more 3.7V cells. Two cells lead down two design paths: a series or a parallel configuration.

A series battery configuration is far more popular so it will be covered first. The immediate upside is batteries in series would provide a 7.4V headroom. Increased headroom would allow more efficient power converters, especially eliminating the need for a boost converter for the motor rail. A "buck only" approach would allow for a much higher power efficiency. Additionally, increased voltage would lead to lower currents which would then lead to lower power loss across resistances in the PCB traces. Both advantages would decrease the power consumed by the batteries leading to longer runtime.

Using batteries in series would require a battery management chip to prevent the battery cells from becoming unbalanced. TI's guide on cell balancing states "Overcharging and overheating of the battery causes reaction of active components with electrolyte and with each other ultimately causing to explosion and fire." Entailing that one cell becoming improperly charged causes an "explosive chain reaction" [15]. Thankfully battery management chips are available to prevent this issue from happening. Below is an example schematic for a multi-series battery pack manager [16].

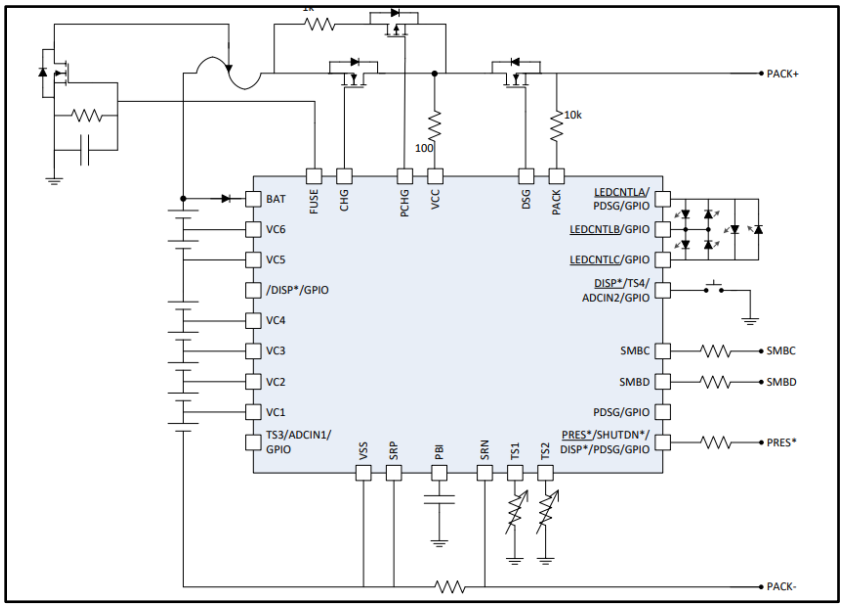

Figure 17: Example Battery Management IC Datasheet

This implementation would make the design much more complex. However, for future implementations when they are more tiles, requiring more power to the motors, this IC would be necessary anyway. So, implementing this chip now, would solve headaches later down the line and prepare the Origami Robot to be expanded to more tiles. The larger issue with adding an additional battery is that the battery would increase the size and mass of our robot, both heavily weighted constraints. This would cause the motors to draw more current and use more power, decreasing the sustainability of the Origami Robot.

A parallel battery implementation is much less popular. The big advantage to running a parallel battery is our previously implemented power management would not need to change as a battery management IC would not be needed since the batteries in parallel inherently have equal voltage. The only other benefit to using battery in parallel is that higher current draw can be higher meaning our battery would last longer, and the chances of the ESP and sensor browning out would decrease dramatically.

This configuration also contributes additional size and mass to the robot, preventing our goal of a small flexible-adjacent robot. The other downside is the batteries in parallel would be difficult to monitor how much current is drawn from each battery, leading to poor battery health over time. For these reasons, the parallel configuration will not be used in either design concept 1 or design concept 2.

Rail Configuration

The rail design for concept 1 that was articulated above is shown below to compare against the second design's rail configuration.

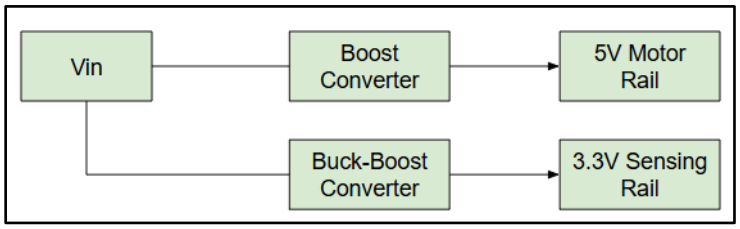

Figure 18: Design Concept 1 Rail System

The rail design for concept 2 is shown below. There are multiple advantages to using this system over the rail concept in design 1.

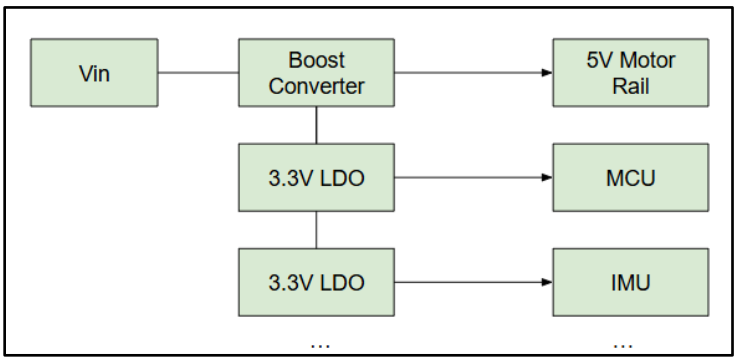

Figure 19: Design Concept 2 Rail System

The first advantage is straightforward: this design would require a less complicated design and save space on the board that is currently being occupied by the buck-boosts configuration. Next, LDO's produce much less noise than switching supplies. In Analog Devices analysis of clean power supply, they state "Furthermore, when it comes to powering noise sensitive analog/RF applications, LDO regulators are generally preferred over their switching counterparts" [17]. In other words, for our IMU and IR sensors, the LDO would greatly reduce the noise caused from the voltage supply. This preventative measure would result in more accurate data being sent to the firmware module. Finally, as the LDO requires less components, each device can be connected to an individual LDO. This factor would greatly simplify the PCB design as eliminating the sensor rail will allow for more flexibility in placing the PCB traces to avoid EMI.

5.1.3 Firmware Module

5.1.3.1 Design Concept 1

In the first design concept of the firmware module, the architecture of the firmware uses three ESP32 microcontrollers in order to handle data transmission between the robot and the Orange Pi.

Communication Specification

The firmware module will use a series of communication protocols in order to propagate the corresponding signals through the communication nodes in the module. The communication protocol that will be used to wirelessly communicate between the first and second ESP32s is called ESP-NOW. Some pros to using ESP-NOW are that there is no handshake or network setup required at all, which allow for instant data transmission between them. With ESP-NOW, the two ESPs communicate directly with each other, eliminating the need to worry about external devices sending data to the ESPs [21]. It also supports expansion for communication with allowing one-to-many and many-to-many communication scenarios as well. If we add more and more tiles onto our design, we would maybe need more GPIO pins resulting in another ESP getting added. Since we are using ESP-NOW, implementing the extra ESP would be extremely easy with the ESP communication protocol [21]. There are built-in encryption methods in the ESP-NOW protocol as well: AES & ECDH Encryptions. Both of these encryption methods are often used together since AES needs a symmetric shared key between the entities and the Diffie-Hellman encryption methods provides secure communication and key establishment in order for the data transmission to occur [22]. The AES and ECDH encryptions complement each other very well allowing the data being transmitted to be safe from external attacks. The second ESP32 will be connected to the Orange Pi via UART. We will utilize the USB UART TTL component, which will help establish a reliable the wired connection without any messy wires going to the Orange Pi. Since the second ESP and the Orange Pi will be extremely close to each other we do not have to worry about the long-distance constraint there is when implementing UART.

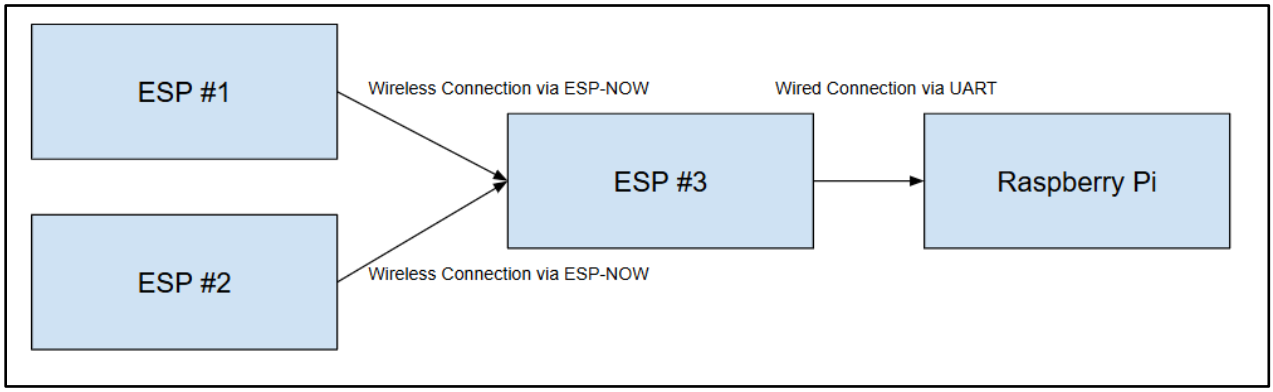

Figure 20: Design 1 Communication Links

Component Specification

The firmware module of the design will be composed of multiple different sensors and cameras. The sensors that will be embedded on the robot's PCB will include an IR Sensor for each of the sides, an IMU sensor, and a Potentiometer for each hinge. There will also be two Arducam Mini Camera Modules on the PCB in order to create a 3D reconstruction of the room via Stereo Vision.

The specific IR Sensor we will be using is the VL53L0X Time of Flight sensor and it will be used to sense any objects that might be in the way of the robot. The ToF sensor uses a laser pulse in order to measure the distance between it and the closest object it detects. It sends out a laser pulse from the sensor and measures the time it takes to bounce the pulse back and does calculations from that data in order to get the total distance it detected. The output of the different IR Sensors will be used in order to give the robot the correct direction it should go from the software algorithm module.

Next, the specific IMU Sensor that we will be using is the MPU 6050 and it will be used to measure the robot's current orientation, motion, and position with the use of accelerometers and gyroscopes embedded on the IMU itself. The IMU will output values for each of the metrics in three axis: x, y, and z and these values will be given to the software module to update its pathing algorithm.

Furthermore, the specific type of split shaft potentiometer we will be using is a 250K potentiometer. The potentiometer will be attached to the hinge axle in order to measure the angle of the hinge when the robot is folding. This will be used to ensure that the angle of each hinge is set to the desired angle given from the software module.

Finally, the type of ArduCAM camera module we will be using is OV2640 2MP mini module. The cameras will be in charge of getting live images of the room they are in and sending the images in the compressed format to the software module. The software module will use the images and the Stereo Vision algorithm in order to create the 3D reconstruction.

Hardware Design

First ESP

The first ESP32 surface mount chip for this design will be embedded on the robot's main PCB and will be connected to the different onboard sensors we have, such as the IR sensors, Potentiometers, and IMUs. This ESP32 will not only be in charge of collecting raw analog data in real-time from the components mentioned above, but also from the camera that will be in charge of taking periodical images of the room. There will be a total of 4 IR sensors connected to the first ESP32 on the robot utilizing the I2C feature of the MCU with the SCL and SDA pins. There will be 5 DC motors connected to the ESP each needing 2 digital pins, and the potentiometer connected to this ESP will be utilizing 1 analog pin. The IMU will also be connected via I2C utilizing the SCL and SDA pins of the ESP32. Finally, the camera will be using SPI and I2C in order to communicate with the MCU by using the MOSI, MISO, CS, CLK, SDA, and SCL pins. The first ESP32 transmits the information obtained from the sensors wirelessly to the third ESP32 using the ESP-NOW communication protocol. The ESP-NOW communication protocol is a built-in wireless protocol for ESP32 microcontrollers and allows direct and efficient communication. The figure of the pinout for the different components used in the first ESP is provided below.

Component	GPIO Pin(s) / Type	ESP #
DC Motor 1	GPIO36, GPIO39 / Digital	1
DC Motor 2	GPIO34, GPIO35 / Digital	1
DC Motor 3	GPIO32, GPIO33 / Digital	1
DC Motor 4	GPIO25, GPIO26 / Digital	1
DC Motor 5	GPIO14, GPIO27 / Digital	1
Potentiometer	GPIO15 / Analog	1
IMU	GPIO21, GPIO22 / I2C	1
IR Sensor 1	GPIO21, GPIO22 / I2C	1
IR Sensor 2	GPIO21, GPIO22 / I2C	1
IR Sensor 3	GPIO21, GPIO22 / I2C	1
IR Sensor 4	GPIO21, GPIO22 / I2C	1
Camera	GPIO21, GPIO22, GPIO5, GPIO18, GPIO19, GPIO23 / SPI	1

Table 1: ESP #1 Sensor Pinouts

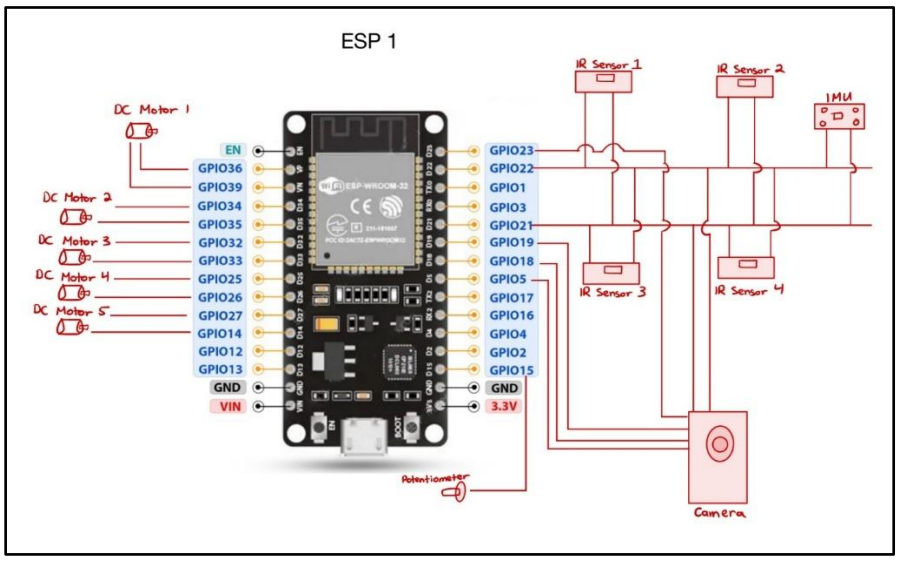

Figure 21: First ESP Pinout

Second ESP

The second ESP32 surface mount chip for this design will also be embedded on the robot's main PCB and will be connected to the different onboard sensors we have, such as the IR sensors, Potentiometers, and a Camera. Similar to the first ESP, the second ESP will be in charge of raw data collection as well as taking images of the environment the robot is in. Again, there will be a total of 4 IR sensors connected to the second ESP32 on the robot utilizing the I2C feature of the MCU with the SCL and SDA pins. There will be 5 DC motors connected to the ESP each needing 2 digital pins, and the potentiometer connected to this ESP will be utilizing 1 analog pin. Finally, the camera will be using SPI and I2C in order to communicate with the MCU by using the MOSI, MISO, CS, CLK, SDA, and SCL pins. The second ESP32 transmits the information obtained from the sensors wirelessly to the third ESP32 using the ESP-NOW communication protocol. The figure of the pinout for the different components used in the first ESP is provided below. The main reason why we need two onboard MCUs is due to the fact that our design requires 42 pins.

Component	GPIO Pin(s) / Type	ESP #
DC Motor 1	GPIO36, GPIO39 / Digital	1
DC Motor 2	GPIO34, GPIO35 / Digital	1
DC Motor 3	GPIO32, GPIO33 / Digital	1
DC Motor 4	GPIO25, GPIO26 / Digital	1
DC Motor 5	GPIO14, GPIO27 / Digital	1
Potentiometer	GPIO15 / Analog	1
IR Sensor 1	GPIO21, GPIO22 / I2C	1
IR Sensor 2	GPIO21, GPIO22 / I2C	1
IR Sensor 3	GPIO21, GPIO22 / I2C	1
IR Sensor 4	GPIO21, GPIO22 / I2C	1
Camera	GPIO21, GPIO22, GPIO5, GPIO18, GPIO19, GPIO23 / SPI	1

Table 2: ESP #2 Sensor Pinouts

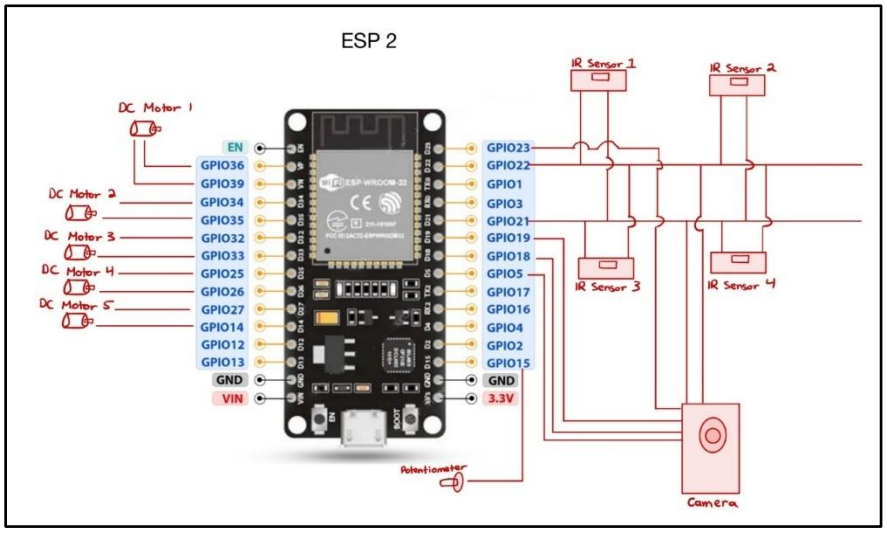

Figure 22: Second ESP Pinout

Third ESP

The third ESP32 is located next to the reconstruction interface and is connected to a Orange Pi via a wired USB connection utilizing UART. The third ESP32 will be embedded on a custom PCB made for it. Since we will be using the ESP32 chip directly for our design, this custom PCB will allow for both of the ESP32s to be programmed without the need for a dev board. The components needed for programming the chip will include an ESP32 SMD chip, 2 push buttons, 2 AA batteries, 10k resistor, and a USB-to-UART TTL adapter. The resistor will be used as a pull-up resistor for the reset button, while the 2 AA batteries will be used as a power supply for the chip itself [20].

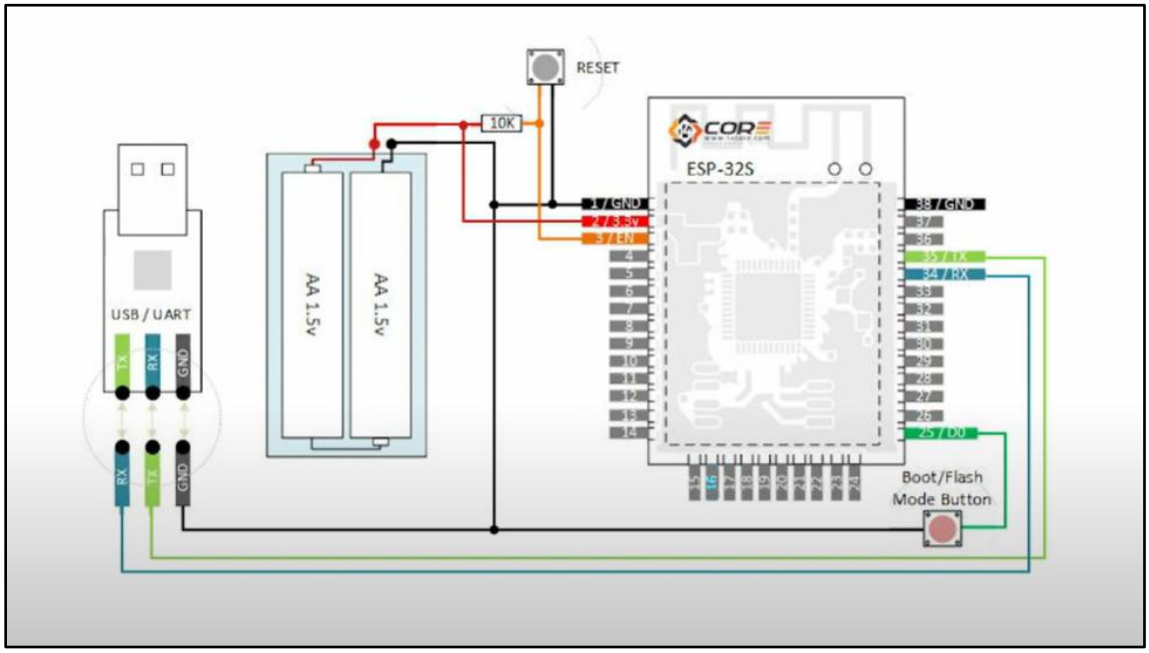

Figure 23: Schematic of Custom PCB for ESP Programming

After constructing the schematic shown above, it is just a matter of plugging in the USB side of the adapter into the Orange Pi and uploading the desired code onto the chip. Before uploading the code on the chip, one important step to follow is to hold down the Boot/Flash button and press the reset button once in order for the Arduino IDE to be able to connect to the chip and upload code successfully via UART. Since the third ESP and the Orange Pi are going to be extremely close to each other, reusing UART communication protocols will be beneficial to avoid any complexity. The custom PCB has a USB-UART TTL component that would allow the third ESP to be directly connected to the Orange Pi for any data that needs to be communicated from the Pi to the ESP, vice versa. The custom PCB will be attached to the Orange Pi via screws. The Orange Pi will be enclosed in a casing that will have screw for the custom PCB to be attached to it physically and directly.

The Orange Pi uses the information obtained from the third ESP32 and processes the data in order to perform intermediate computations and run the algorithm that will be in charge of the 3D reconstruction of the room. The hardware of this design is done in such a way that it will allow the project to be portable and have the ability to be used in any of the places that it needs to be.

Software Design

The firmware that will be running on the first ESP32 will handle data collection, sensor polling, and sending the data obtained to the second ESP32 in data packets via wireless communication. The sensors that will output data to the first ESP32 will include IR sensors, Flex Sensors, IMUs, and Cameras as well. An IR sensor will be on each of the sides of the robot in order for the environment that the robot is in to be known and not bump into anything. The IR Sensor will be coded in a way that will output the distance between the sensor and the closest object that it detects. The way movement will work in our system is that the firmware on the robot will be constantly sending the software module a Boolean value that will represent if the robot is able to move forward or not. If any of the IR sensors detect an object 50 millimeters away, then the Boolean variable sent to the software module will be false (it is default to true). The IMU embedded on the robot will be used to measure the robot's orientation and acceleration by combining data from accelerometer and gyroscopes. The IMU will be placed directly in the middle of the robot in order to get the most accurate readings. The IMU readings will be sent to the software module in order for the algorithm running on that module to properly calculate its exact position and its previous positions as well. The main purpose of the IMU is for the software module to know where the robot is and map out the room as accurately as possible. Furthermore, there will be two cameras placed on the robot in order to take periodically images of the environment the robot is placed in. The images taken both cameras will be compressed into a JPEG format and have a resolution of 320 x 240. Compressing the image and lowering the resolution will ensure that the data transmission will not result in unnecessary delays, but still result in an accurate representation of the environment it is currently in. Finally, the flex sensor will be used in order to measure the angle of each hinge of the corresponding tile. The sensor will be placed so that one end will be placed on one tile and the other end of the sensor will be placed on the connecting tile. The main job of this sensor is to start and stop the motors that actuate the hinges depending on the amount the flex sensor has flexed.

The firmware on the second ESP32 will handle receiving the previously mentioned data packets, error checking them, and formatting them in a way that will allow for the Orange Pi to use the data without having to format it itself. The second ESP32 makes sure that the data being forwarded to the Orange Pi is formatted correctly and would not cause any errors when inputted into the algorithm that is in charge of the 3D reconstruction of the environment. The error handling of the data will include making clear error responses, addressing errors at earliest stages, ensuring consistent data types, and using exceptions. One of the ways the firmware is going to handle errors is by making the transmitter resend the data to the receiver. This will make sure that no data got corrupted during transmission that caused the error in the first place. As mentioned above, the second ESP32 will forward the data to the Orange Pi through a wired link via UART. On the Orange Pi, the software will run functions that will mutate the data into what the interface running the Structure from Motion exactly wants it formatted as. This will reduce the number of computations that the interface would need to do resulting in an efficient hierarchy of communication.

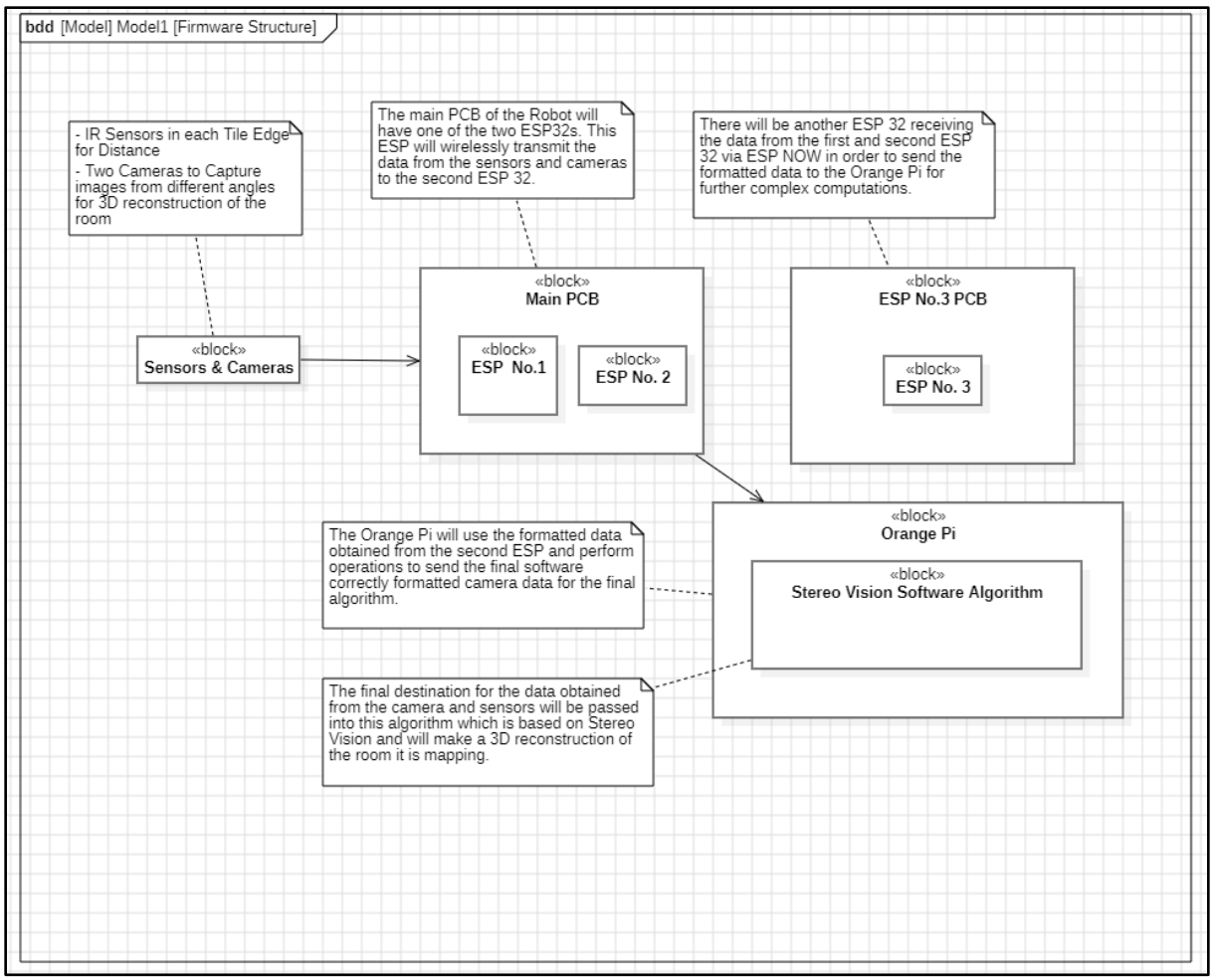

Figure 24: Design Concept 1 Block Diagram

5.1.3.2 Design Concept 2

In the second design concept of the firmware module, the architecture of the firmware uses three ESP32 microcontrollers in order to handle data transmission between the robot and the Orange Pi. The main difference between the first and second Design Concept is that the second Design Concept will only involve two ESP32 microcontrollers instead of three. The majority of Design Concept 1 will apply to Design Concept 2 as the majority of components are the same.

Number of ESPs

As mentioned above, Design Concept 2 will only involve two ESP32 microcontrollers instead of three. In this design, the first ESP32 will still be embedded on the robot's main PCB and connected to the different sensors onboard, such as IR sensors, IMU, potentiometers, and cameras as well. It will be responsible for data collection, error handling, sensor poling, and packaging of the data packets.

Both of the ESP32s will wirelessly transmit data to the Orange Pi using ESP-NOW, similar to the first Design Concept. Unlike Design Concept 1, there will be no wired communication occurring in this design. The ESP microcontrollers will be collecting data from the exact same sensors as Design Concept 1.

The tradeoff for this Design Concept is that it relies solely on the ESP microcontroller to both collect the data and error check it as well. This may introduce an increase in computation and may result in bottlenecks as well compared to the three-ESP Design Concept. But, with efficient firmware and software design, these tradeoff risks can be minimized.

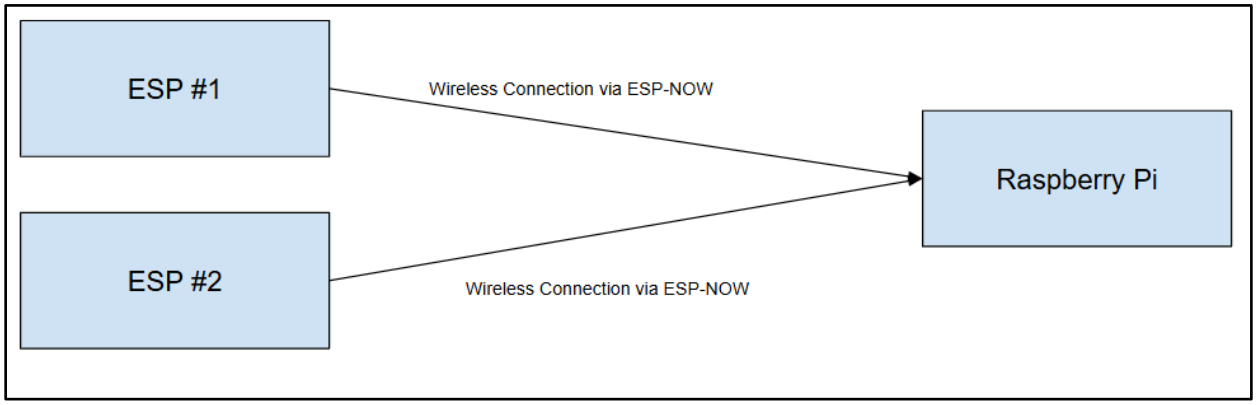

Figure 25: Design 2 Communication Link

The block diagram below shows on overview of how the two ESP MCUs will gather data from the sensors and camera and format them in a way that will allow the Orange Pi to directly use the data for the 3D reconstruction algorithm. Both ESP32s use the ESP-NOW protocol to wirelessly transmit data packets directly to the Orange Pi. The Orange Pi then acts as the node that receives the data from both ESPs and runs the higher-level algorithms. By removing the 'middle man' ESP we had in Design Concept 1, it simplified this design, while still making sure that all of the necessary data is propagated throughout the network.

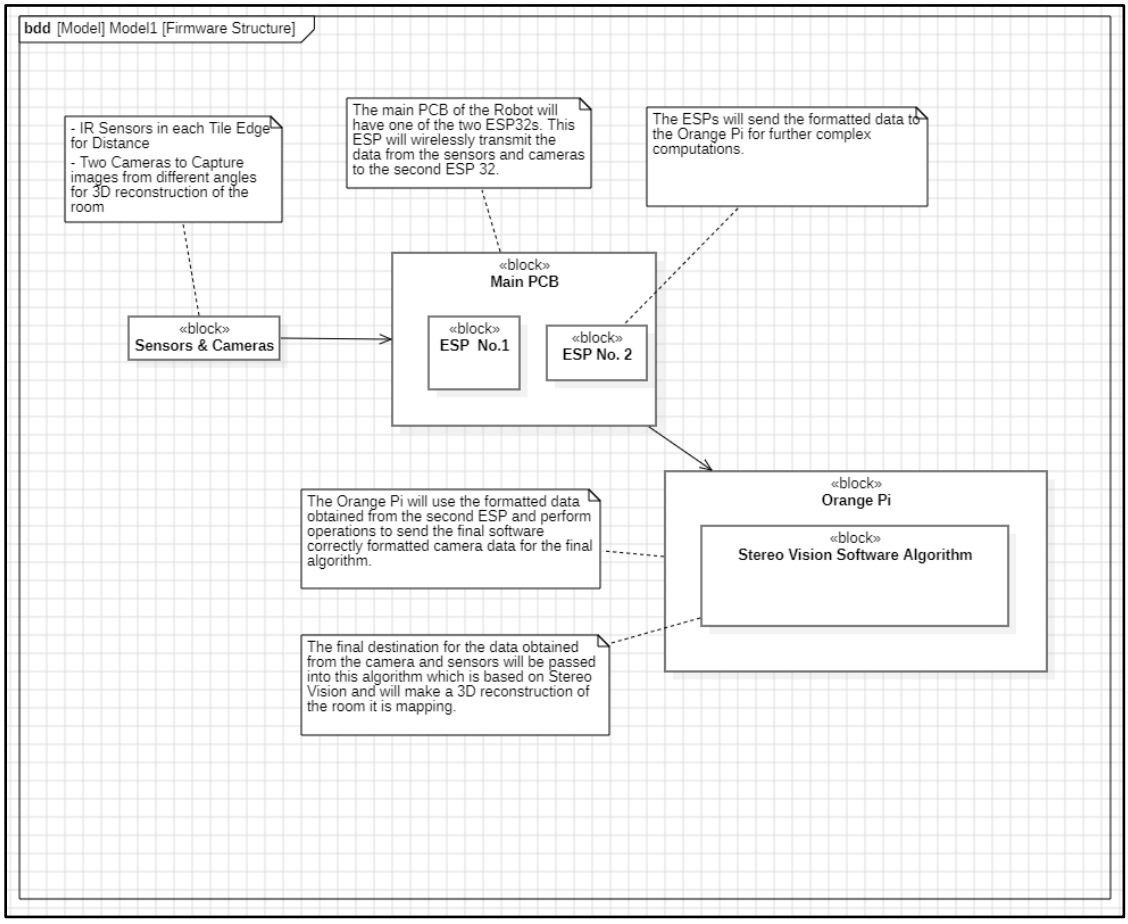

Figure 26: Design Concept 2 Block Diagram

5.1.4 Software Module

The software module will feature designs to localize and map the robot's environment in real time, generate accurate 3D reconstructions, execute folding sequences, and allow for manual operation of the robot. This will require integrating various open-sourced libraries for SLAM and 3D visualization using the ROS (Robot Operating System) framework.

5.1.4.1 Design Concept 1

SLAM

The SLAM (Simultaneous Localization and Mapping) algorithm provides the robot with real-time localization and sparse mapping capabilities. The library ORB-SLAM3 will be used to track visual features and estimate scalable depth for landmarks, which enables pose estimation and loop closure (returning to a previously visited location). The stereo camera feed serves as the primary input, along with additional data from an IMU (inertial measurement unit). The outputs include the robot’s position and orientation in the form of coordinates within the "world". This pose data is essential for autonomous navigation and will be utilized during the 3D reconstruction of the environment. Also, the algorithm will send movement commands to the robot on how to navigate the space.

3D Reconstruction

The 3D reconstruction algorithm builds dense volumetric models of the environment using the open-sourced library, InfiniTAM. The primary inputs include the stereo camera feed and robot poses (from ORB-SLAM3). This data is then processed into a TSDF (truncated signed distance field) volume, generating a consistent 3D reconstruction. This volume can then be visualized using the ROS tool RViz.

Folding Instructions

The folding algorithm governs the robot's transformation into different shapes. Since this project acts as a proof of concept for a folding robot, these shapes will be based on a specially designed obstacle course. This allows the folding patterns to be pre-defined in a JSON file which will specify a sequence of hinge actuations for each shape. As the robot navigates this obstacle course, it will use machine learning and image processing techniques to determine which shape best fits the current obstacle in its path.

Manual Operation

The software will provide the user with an option to assume control of the robot and move it throughout the space. This will require the user have a live feed of the robot's camera and a controller to give the robot commands. Furthermore, using IR data from the robot, the software will be able to notify the user if a collision in imminent and from which direction.

Hardware

The software will be running on an Orange Pi 5 Pro. This will allow the InfiniTAM library (OpenCL implementation) to utilize the Pi's support for GPU acceleration using OpenCL, thus leaving the Pi's CPU to handle the less computationally intensive SLAM and folding algorithms.

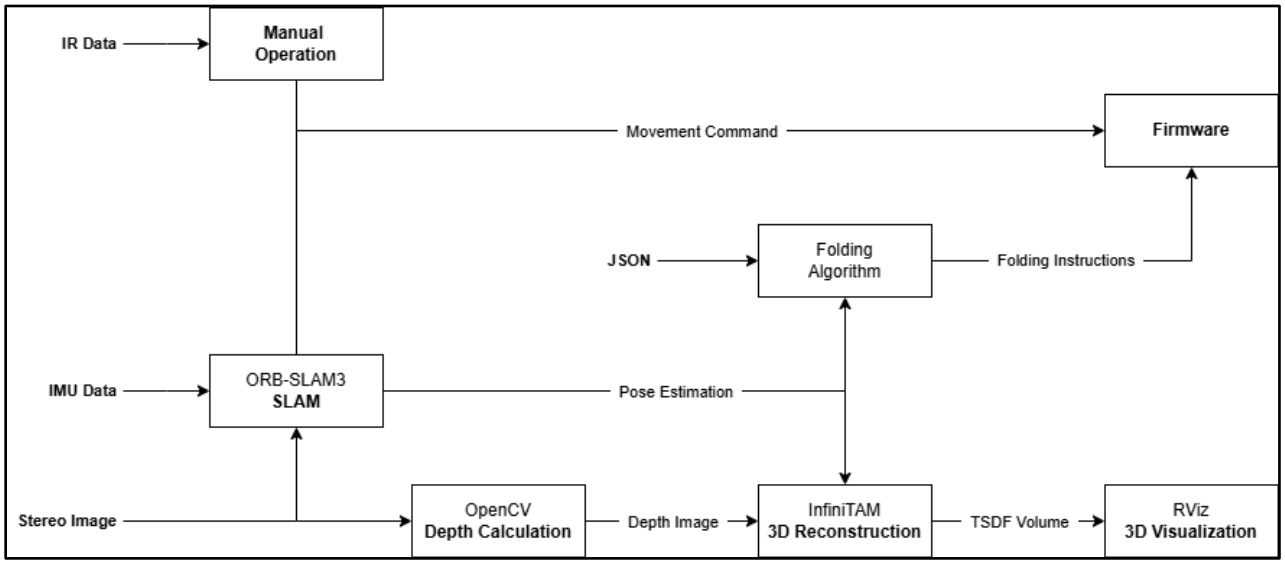

Figure 27: Software Design Concept 1 Block Diagram

5.1.4.2 Design Concept 2

This alternative to the first design concept is mostly the same except it utilizes Open3D instead of InfiniTAM & RVis for 3D reconstruction and visualization.

SLAM

Same as design concept 1.

3D Reconstruction

The 3D reconstruction algorithm builds dense volumetric models of the environment using the open-sourced library, Open3D. The primary inputs include the stereo camera feed and robot poses (from ORB-SLAM3). This data is then processed into a TSDF (truncated signed distance field) volume, generating a consistent 3D reconstruction. This volume can then be visualized using tool from Open3D.

Folding Instructions

Same as design concept 1.

Manual Operation

Same as design concept 1.

Hardware

The software will be running on a Nvidia Jetson. This will allow the Open3D library to take advantage of the Jetson's CUDA cores for GPU acceleration. This will allow for smoother 3D reconstruction compared to running InfiniTAM on an Orange Pi 5 Pro.

5.2 Selected Design Concept

This section will acknowledge and explain the specific design concept each of the corresponding leads decided to pick for their module.

5.2.1 Selected Controls Design Concept

The Controls Lead has decided to pick the first design concept for the motor driver hardware and mechanical hinge assembly. For the control loop implementation, aspects of both design concepts will be used to create a comprehensive control system for both the hinge and translational motion of the robot. The DRV8833 motor driver and N20 DC motor actuated hinge should provide a robust and reliable system from which to build on. The hardware and mechanical aspects of the robot are among the most important, as the control software later on relies on a predictable system with low disturbances. PID control will be used to tune the hinge actuation, and a more complex control loop may be used for positional control.

5.2.2 Selected Power Design Concept

The Power Lead has decided to pick the first design concept using only one LiPo battery and two separate rails set at the voltages of 3.3V and 5V. This concept allows the Origami Robot to stay lighter weight, which is more useful for complex dynamic rigid environments where heavy weight would degrade performance. It would keep the cost of manufacturing the robot lower as money would be saved on the extra battery and the additional parts

required to balance multiple batteries. However, an important caveat to note is if the motors are measured and found to perform with significantly better efficiency under 7.4V, two batteries will be used. The two-rail system is chosen because the large number of devices on the 3.3V will draw high levels of current that need to be transformed efficiently. LDO's would prevent noise from interfering with the sensors but adding them to each device would become expensive and inefficient. Again, if the noise from the 3.3V interferes with the sensors and data mentioned, then the section design will be chosen.

5.2.3 Selected Firmware Design Concept

The Firmware Lead has decided to go with the first design concept involving three ESP 32s as the selected design concept for this module. This concept allows there to be two ESP32s to handle communication with the different sensors onboard, while the other ESP handles data formatting, error-checking, and transferring data to the Orange Pi. This is a good idea because it helps reduce processing load and any chance of bottlenecks in the pipeline. It also increases data transmission reliability, since the ESP-NOW protocol being used is directly developed by the makers of the ESP32 chip. The ESP32 MCU is chosen due to its low power consumption and built-in Wi-Fi/Bluetooth capabilities to help communicate with one another. It also allows for there to have modularity, where each MCU can be programmed and tested independently, allowing for easy implementation and updates as well.

5.2.4 Selected Software Design Concept

The Software Lead has decided on the first design concept involving the InfiniTAM library running on an Orange Pi 5 Pro. This was chosen because the team is already in possession of the Orange Pi and acquiring a Nvidia Jetson would be too expensive. Therefore, design concept 1 is the most feasible of the two and still allows for effective processing of data and real-time operation.

5.3 Sustainability Considerations

5.3.1 Control – N20 DC Motor Selection

The motors selected for this project were selected because they were the most inexpensive option that still met the torque requirements for their application. Sustainability was not considered in this decision, and already 3 of the 10 motors were defective. For future iterations of the robot, more emphasis could be placed on sustainable components rather than the cheapest option.

5.3.2 Control – Material Optimization

When making prototypes and in the final PCB, cutouts may be added to the tiles to reduce weight of the robot. These empty spaces will be used to make motor brackets, preventing unnecessary waste

5.3.3 Power – Rechargeable Battery

Sustainability is built into the design of the Origami Robot; a rechargeable battery will reduce the waste created from using this design. The charge of the battery will limit the current flow into the battery after full charge to reduce unnecessary power consumption. The battery will also be monitored for voltage sag to prevent damage done to the battery, leading to unnecessary battery replacements.

5.3.4 Power – High Power Efficiency

Buck-boost and boost converters will be chosen to supply voltage to the loads at efficient states to reduce the power consumption through thermal loss. Protections such as brownout detection will limit unnecessary current draw.

5.3.5 Power – PCB Layout

The PCB will be laid out to prevent excessive waste. This will be done by minimizing cutouts in the PCB, and keeping the tiles small in width and length.

5.3.6 Firmware – Low Power Consumption

By taking advantage of the ESP32's low power consumption, the firmware will be designed to minimize energy consumption either when the robot is not doing anything or when certain sensors are not actively being used. This also helps extend the operational time of the system without having to buy large batteries.

5.3.7 Firmware – Algorithm Optimization

The firmware that will reside in the MCU(s) will rely on efficient sensor polling rates, communication protocols, and error-handling function in order to reduce the need for unwanted computation. This helps the firmware to reduce the CPU usage and the amount of power drawn to the MCU(s).

5.3.8 Firmware – Modular and Scalable Code

The firmware in the MCU will be written in such a way that future expansion will not be a problem. Future expansion might include adding some sensors or other communication features to the system without having to rewrite the entire code base for it. This helps the robot to adapt to any changes that need to be in the future in order to keep the robot up and running.

5.3.9 Software – Algorithm Optimization

The software will run, in real-time, on a regular sized SBC like an Orange Pi or others like it. Writing efficient algorithms not only reduces the computational complexity of the software, but also reduces CPU and, if applicable, GPU usage. This results in a lower overall power draw of the SBC.

6. System Test and Verification

This section will include test plans for each of the module in our system design. The test plans will consist of three different categories: Performance, Software, and Hardware. Throughout the process of our implementation, we will be conducting tests on specific modules in order to ensure that individual modules work before integrating them together.

6.1 Performance Testing

6.1.1 Controls – Control Loop Performance Parameters

Hinge control performance will be measured based on rise time, settle time, overshoot, and steady state error. These parameters will be measured by collecting data from the feedback sensor and graphing these values against time. By overlaying the setpoint step input, the performance parameters may be measured and optimized by tuning the control system gains.

6.1.2 Controls – Hinge Range of Motion

Range of motion of the hinge will be measured by finding the maximum angle at which the actuating tile can move to relative to the plane of the adjacent tile. This angle should be 90 degrees in either direction, allowing the actuating tiles to be used to effectively navigate obstacles.

6.1.3 Power – Methods

The PCB will be designed with pin headers for battery voltage, the voltage seen by the converters, the rail voltages and GND. This feature will allow for simple testing methods that are listed below.

6.1.4 Power – Runtime

As mentioned above, the runtime of robots should be over an hour. Runtime implies that all of the systems on the robot will work properly for an hour, this includes IMU performance, ESP performance and motor performance. We will test and verify this by monitoring the current and voltage levels during testing. There must not be any brownouts for the ESP, which can simply be seen if the ESP turns off. The battery voltage must be over 3V during testing, and current draw must be under 2A. The hinge angle and control must be able to settle properly in the control loop and improper voltage will cause issues with the control. Finally, the MCU bandwidth must not drop during the hour runtime.

6.1.5 Firmware – Latency

As mentioned above, the firmware shall make sure that the data transmission from the first ESP to the Orange Pi occurs within 50 milliseconds. The test case will consist of measuring end-to-end packet transmission time using timestamped logging at each of the ESPs and Orange Pi. The expected outcome of this test case is there to be an average transmission latency that is less than or equal to 50 ms under normal operating conditions. The equipment needed to test this are three ESP32s, Orange Pi, sensors, software logging for timestamp comparison.

6.1.6 Firmware – Reliability

As mentioned above, the firmware shall have the ability to achieve a successful packet transmission rate of 95% throughout the operation of the Origami Robot. The test case will consist of transmitting a known sequence of data packets over multiple trials and calculate the percentage of packets that were received successfully. The expected outcome of this test case is there to be more than 95% of the packets are received correctly throughout the trials. The equipment needed to test this are three ESP32s, Orange Pi, sensors, software logging for stream of bits transmitted.

6.1.7 Firmware – Bit Error Rate

As mentioned above, the firmware shall have the ability to achieve a bit error rate of, such as 10^{-6} throughout the operation of the Origami Robot. The test case will consist of sending a large dataset from the ESP32 to the Orange Pi and count the incorrect bits transmitted versus the total bits transmitted. The expected outcome of this test case is there to be a bit error rate of less than 10^{-6}. The equipment needed to test this is three ESP32s, Orange Pi, sensors, software to compare received vs. transmitted data.

6.1.8 Software – Rendering Framerate

As mentioned above, the software shall be able to create a 3D visualization of its environment. This involves heavy computations which limit how fast the software can process images that are received from the robot. For reliable navigation and 3D visualization the software should be able to process at least 10-15 frames per second. The test case will consist of using a free-to-use game engine, such as Unreal Engine or Unity, to create a virtual environment with a virtual camera feed. This feed will then be passed into the software and then the operating framerate will be measured during runtime.

6.1.9 Software – Rendering Quality

As mentioned above, the software shall be able to create a 3D visualization of its environment. This requires the configuration of various parameters to detect and eliminate junk data and excess noise. The test case will consist of using a free-to-use game engine, such as Unreal Engine or Unity, to create a virtual environment with a virtual camera feed. This feed will then be passed into the software where these parameters will be adjusted based on a visual inspection by the user.

6.2 Software Systems

6.2.1 Control – Control Loop Implementation

A PID control loop will be implemented in software as a C++ class. This will integrate with the firmware modules to collect data from the feedback sensors and compute new setpoints for the hinge angle. This software will also consider practical non-idealities of physical control systems, such as the kick produced by the derivative term and integrator windup. To account for this, limits will be set on the controller output and a sufficiently high sample time will be used.

6.2.2 Power – Brownout Detection

To see if the ESP will reset under brownout conditions, a load will increase on the sensing rail until the brownout is reached. Then if the ESP firmware resets, the brownout protection is validated.

6.2.3 Firmware – Algorithm Testing

The algorithm running on the firmware for data collection, forming data packets, and error handling will be tested to ensure that they perform as expected. Testing these features include verifying that the sensor output data is correctly collected and sent to the third ESP32 in organized data packets such that it can be decoded on the third ESP as it will be programmed to do. The algorithm on the firmware will encode the data from the sensors and the camera in a way such that it will result in less transmissions to get all of the data to the Orange Pi and organized such that parsing the data to decode it will be easy as well. Error handling will be present at places where encoding and decoding occurs, since we want the data packet structures to be identical to one another during normal operation. The error handling will include retransmission of the data packet and collecting the sensor and camera data again as well.

6.2.4 Firmware – Integration Testing

Testing the firmware module with the modules that it is interacting with and their corresponding input/output will verify that the firmware module is functioning as intended. Integration testing for the firmware module will include sensor interface, wireless communication, wired communication, and data packet management. These tests will prove that the different features in the firmware will work together seamlessly to transmit data to the software side of the system and work together with the power side of the system as well while checking that data flows correctly without packet loss or delays.

6.2.5 Firmware – Unit Testing

Each of the components that collectively make up the firmware module will be tested independently in order to make sure correct functionality. The sensor interface will be tested to make sure for accurate data collection. The communication module will be tested for correct data packet encoding and transmission for each of the communication node present in the firmware module of the system.

6.2.6 Software – SLAM and 3D Reconstruction Testing

The SLAM and 3D Reconstruction algorithms will go through two sets of tests. The first will have them running independently of each other to ensure they are working as intended. The next set of tests will have them work in tandem to navigate through a space while generating a 3D visualization of the space. This will be done by using a free-to-use game engine, such as Unreal Engine or Unity, to create a virtual environment with a virtual camera feed. These algorithms will then use this camera feed to complete their respective tasks.

6.2.7 Software – Integration Testing

Testing the software module alongside the firmware module and monitoring their inputs/outputs will verify that the software module is functioning as intended. This includes receiving sensor data from the firmware, processing the data, and sending the correct commands back to the firmware module.

6.3 Hardware Systems

6.3.1 Control – Motor Drivers

Since this robot will be using many N20 DC motors, a robust motor driver design is imperative to the project's success. A DRV8833 chip will be used to control two motors, as it is a dual H-bridge design. Bulk capacitance should mitigate any voltage drop caused by motor inrush current. While driver boards will be used in testing and initial prototyping, the driver components will be designed into the overall PCB in strategic locations to minimize overall footprint and optimize data signal integrity. The Controls Lead will work with the Power Lead to integrate this hardware assembly into the PCB.

6.3.2 Power – Charging Module

The power system is to be tested when the USB-C is connected and when it is not. When the USB-C is attached, the voltage seen by the converters should be 5V and when the USB-C is plugged in and out, there should be a minimum about of voltage over and undershoot. When the battery is the only source, the voltage seen by the converters should be around 3.7V.

6.3.3 Power – Signal Integrity

The power system is to be tested for EMI through using a motor at its operating frequency. The IMU, camera and ESP voltage pins will be checked to see if the 3.3V is still clean.

6.3.4 Firmware – Verify Sensor Signal Output

Each of the sensors embedded on the robot and connected to the first ESP 32 will be tested to confirm that the output signals match the expected values and format of those values. This is tested to make sure that the sensors have been integrated onto the PCB correctly and the PCB is propagating those signals properly as well.

6.3.5 Firmware – Verify PCB for third ESP32 is working as intended

Check that the custom PCB made for the third ESP 32 correctly interfaces with the Orange Pi and is powered by the Orange Pi itself. Tests will include correct data packet formation for the transmission to the Pi and measuring the performance of the transmission as well.

6.3.6 Firmware – Verify Connection between Communication nodes work as intended

Both the wireless link between the first/second ESP32 and the third ESP32 and the wired link between the second ESP32 and the Orange Pi will be tested in order to ensure data is getting propagated through the nodes properly. The connection between the third ESP32 and the Orange Pi will be done via UART, which means the TX of the ESP must be directly connected to the RX of the PI, vice versa.

7. Team

This section will include an introduction to all of the team members for this project and their corresponding background that will allow them to contribute to their role.

7.1 Karthik Raja

Karthik is a Computer Engineer pursuing an autonomous system concentration along with the Cybersecurity certificate. He will be responsible for the development of the firmware of the Origami Robot, including the microcontroller programming, sensor integration, motor control logic, and communication between the robot and the MCUs. He has experience in software engineering (Python, C/C++ for embedded systems and data handling) and embedded systems (ESP32/STM32 programming, serial and wireless communication, and hardware interfacing), which will be applied to this project.

7.1.1 Skills learned in ECE coursework

Karthik has taken ECE 1150 (Intro to Computer Networks) and ECE 1155 (Information Security), which provided a strong foundation in both wired and wireless communication protocols. In both of these courses, he learned the basics of serial communication, error detection methods, and the best practices for reliable data transfer between systems. All of these skills will be applied when implementing communication between the Origami Robot's microcontroller and other microcontrollers or an Orange Pi.

He has also taken ECE 0202 (Embedded Processors and Interfacing) and ECE 1175 (Embedded Systems Design), both of which introduced the basics of embedded systems, including microcontroller programming, peripheral interfacing, and hardware-software integration. ECE 1140 (Systems and Project Engineering) also provided Karthik with the process of hardware-software integration. These courses provided hands-on experience in working with digital logic and sensor input/output, which are essential for programming the Origami Robot's motor movement and sensor integration.

7.1.2 Skills learned outside ECE coursework

Outside of ECE coursework, Karthik gained valuable experience as a Software Engineering Intern at Keystone Collections Group, where he learned version control via GitHub and managing previous versions of software properly. Also, during this internship, he also practiced the best software engineering skills, such as writing modular and scalable code, documenting his work, and writing unit tests to ensure all components of system work.

This semester, Karthik plans to gain knowledge specifically for this project by researching the specific camera module we plan to use and understanding USB-to-UART TTL connections via datasheets and online tutorials. All of these skills directly support the firmware requirements of the Origami Robot.

7.2 Jacob Ferrer

Jacob is an Electrical Engineer pursuing an integrated circuits concentration. He will be responsible for the development of the power circuitry and PCB design of the Origami Robot, including the battery control system, protection, and efficiency of the robot's power system. He has experience in PCB design (KiCad, Altium) and integrated circuit design (power modules, microelectronics), which will be applied to this project.

7.2.1 Skills learned in ECE coursework

Jacob has taken ECE 1212 (Electronic Circuit Design Lab) and ECE 1895 (Junior Design Fundamentals), which provided a strong foundation in both analog circuit design and PCB design. In both courses, he learned the basics of multistage amplifiers, power pathing, and PCB design best practices. All these skills will be applied when implementing the power electronics system in the Origami Robot.

He has also taken ECE 1890 (ECE Prototyping Fundamentals) which introduced the basics of soldering, and PCB layout best practices. This course provides practical experience that can be applied when the final origami is being built. Finally, Jacob has taken ECE 1710 (Fundamentals of Electric Power Engineering), and while the course taught mainly power systems at a grid level, the course gave Jacob a foundation on power ideas and power conservation practices, as well an introduction to power protection at a grid level. These courses provide a foundation for Jacob to construct the power system and PCB of the Origami Robot.

7.2.2 Skills learned outside ECE coursework

Outside of ECE coursework, Jacob gained valuable experience as a Power Delivery Engineering Intern at Kimley-Horn, where he learned power system protection schemes as well as high level power tree design. Also, during this internship, he also practiced the best electrical engineering skills, such as quality assessments and checking simulation solutions with pen and paper evaluations.

This semester, Jacob plans to gain knowledge specifically for this project by researching the best battery module control system, power tree design, and advanced PCB design techniques to mitigate power losses. Additionally, he will reference professional guides from companies such as TI and Analog Devices as well as sourcing power electronics textbooks from Mohan, Erickson and Maksimović.

7.3 Will Roper

Will is an Electrical Engineering student pursuing a power concentration, with additional emphasis on control systems. He will be responsible for designing and producing the motion systems within this project. This includes the motor drivers, control loop, necessary feedback sensor integration, and some mechanical design of the hinge assembly. He has experience with prototyping and fabrication techniques as well as practical applications of control loops in a previous project.

7.3.1 Skills learned in ECE coursework

Will has taken ECE 1674 (Mechatronic Systems), ECE 1673 (Linear Controls), and is currently taking ECE 1771 (Electric Machinery). These courses have provided him with extensive knowledge of motors, drivers, and motion control systems in both a theoretical and practical context. He has completed the Mechatronic Systems labs, which let him test motion control loops and measure parameters to determine optimal tuning. These skills will be very helpful in both the design and execution of the motion control portion of the robot.

He has also taken ECE 1895 (Junior Design Fundamentals) and ECE 1890 (ECE Prototyping Fundamentals) which gave him experience designing, ordering, and troubleshooting PCBs. This knowledge will be helpful when designing the motor drivers and integrating them into the overall PCB.

7.3.2 Skills learned outside ECE coursework

Will has diverse industry experience outside of class in professional environments. He has completed two internships with Eaton Corporation in their Electrical Systems and Services division, giving him necessary skills to manage project timelines and interface with other professionals or stakeholders. He has also had accountability over a summer project in each of these internships, building self-starting skills and learning difficult concepts through research.

Several projects have offered Will technical skills that will be useful in the execution of his role in the Origami Robot project. He built an inverted pendulum, which used a PID control loop to stabilize a vertical rod on a free-spinning pivot. This project combined mechanical and hardware integration with a software-based control loop, so it was an ideal precursor to a project such as this one. He has also worked with First Energy to produce a smart voltmeter capable of plugging into an outlet. This project provided him with electronic prototyping skills and team management.

7.4 Zachary McPherson

Zachary is a Computer Engineer pursuing a concentration in Autonomous Systems. He will be responsible for the development of the software of the Origami Robot, including the implementation of SLAM, 3D reconstruction, folding sequences, and manual control. He has experience in software engineering (C/C++/C#, Python) and embedded systems (ESP32, STM32, & SBC programming, serial and wireless communication), which will be applied to this project.

7.4.1 Skills Learned in ECE Coursework

Zachary has taken ECE 0202 (Embedded Processors and Interfacing) and ECE 1175 (Embedded Systems Design), both of which introduced the basics of embedded systems, including microcontroller programming, the handling of data from peripherals, and hardware-software integration. ECE 1140 (Systems and Project Engineering) also provided Zachary with the experience of integrating multiple software modules and designing the communication link between them. He has also taken ECE 1895 (Junior Design Fundamentals) which gave him experience in developing software for SBCs such as Raspberry Pi and Orange Pi.

Currently, Zachary is taking ECE 1390 (Image Processing) and ECE 1395 (Intro to Machine Learning) which introduce the basics of processing image data and machine perception. These skills will prove useful in the implementation of SLAM, 3D reconstruction, and how the robot decides when to change its shape.

7.4.2 Skills Learned outside ECE Coursework

Zachary has industry experience working as a software engineering intern at Alstom where he learned how to design dynamic software through the use of JSON files and the integration of this software into vital systems. These skills will prove useful because the Origami Robot will get its folding instructions from JSON files and will require the integration of SLAM and 3D reconstruction algorithms.

8. Schedule and Budget Plan

8.1 Project Schedule

Changes to the budget are **bolded** in the schedule.

Week 09/29:

- Will: Prototype hinge mechanism with feedback sensor, motor, and motor driver.
- Jacob: Test current and pull from set voltages for all devices, choose proper parts **order devices.**
- Zach: Install and setup ROS, RVis, ORB-SLAM3, and InfiniTAM libraries.
- Karthik: Make a breadboard prototype with all sensors and ensure they output valid data to the ESP32.

Week 10/06:

- Will: Refine control loop to achieve desired hinge angles with prototype. Begin motor driver circuit design. Work design for PCB tile layout and complete PCB for 2nd ESP32 chip.
- Jacob: Finish the initial PCB design, have it printed. **Order PCB.**
- Zach: Begin implementing basic SLAM and 3D reconstruction algorithms as well as reading JSON folding instructions.
- Karthik: Implement and test sensor firmware drivers (IMU, distance sensor, camera). Verify data reads consistently. Add basic GPIO test program to toggle motor control pins (simulate forward/backward/hinge movement).

First Checkoff Expectation – 10/08:

Significant module progress and at least one quantitative test case

- Firmware demo: Demonstrate working sensor → ESP32 communication and GPIO toggling for motors.
- Controls demo: Functional Hinge mechanism and wheels with PID control loop (include control parameters)
- Power demo: Completed PCB design (Basic Calculations completed)
- Software demo: Demonstrate 3D reconstruction with set of stereo images.

Week 10/13:

- Will: Integrate wheel functionality to hinge prototype. Explore control loops using IMU feedback. Finish and submit the custom PCB from last week.
- Jacob: Develop further protection schemes for the power system as well as switching supply design.
- Zach: Continue implementing basic SLAM and 3D reconstruction, as well as installing and setting up Unity testing environment.
- Karthik: Develop and test ESP-NOW communication between the two ESP32s (send sample sensor data and motor command signals).

Week 10/20:

- Will: Work with software and firmware leads on acceleration (motion) control loop.
- Jacob: Put together PCB, with devices (soldering).
- Zach: Run basic algorithms independently through Unity testing environment.
- Karthik: Implement wired connection (UART/I2C/SPI) between third ESP32 and Orange Pi. Verify end-to-end data pipeline: sensors → ESP32 1/2 → ESP32 → Pi, and Pi → ESP32 motor commands.

Midterm Presentations – 10/22:

- Firmware status: Show working data transmission and motor signal reception.
- Power status: Show working PCB (ESP, Motors, Sensors, Battery Charging)
- Control status: Show working laser-cut prototype with hinge actuation and translational motion.
- Software status: Show timelapse of Unity test environment.

Week 10/27:

- Will: Build full-scale prototype with laser cut tiles or initial PCB.
- Jacob: Benchmark test PCB for functional requirements. Verify sensor and power rails are constant. Measure current draw. Verify charging is working properly and efficiently. Sensor and power rails to be used by sensors and motors. Document test results.
- Zach: Integrate SLAM, 3D reconstruction, and folding algorithms.
- Karthik: Define packet structure for sensor data + motor commands. Add error checking and acknowledgment for reliability.

Week 11/03:

- Will: Test turning radius and experiment with folding commands to navigate obstacles.
- Jacob: Fix any errors found in the PCB. Fix any errors found through module testing. Implement additional protection schemes. 2nd and final PCB design. **Order of PCB and additional parts.**
- Zach: Begin software integration with firmware. Software is able to receive sensor data from firmware and firmware is able to receive commands from software.
- Karthik: Begin firmware integration with software motion algorithm. Commands generated by Pi are received by ESP32 and translated to GPIO signals for motors/hinges.

Week 11/10:

- Will: Address integration issues, fine-tune control loops for accurate motion.
- Jacob: Begin work on final paper documentation.
- Zach: Finish integration of SLAM, 3D reconstruction, and folding algorithms.
- Karthik: Conduct unit tests on each firmware module (sensor drivers, wireless comm, wired comm, motor GPIO signaling). Document test results.

Second Checkoff Expectation – 11/12:

- Firmware milestone: Show reliable bi-directional pipeline: sensor data → Pi, motor command → motor pins.
- Software milestone: Show timelapse of Unity test environment with full algorithm integration.

Week 11/17:

- Will: Begin documentation, prepare parts for final assembly.
- Jacob: Build and test final PCB. Ensure modules are working properly together.
- Zach: Perform system-level testing of software under realistic robot operation.
- Karthik: Perform system-wide testing of firmware under normal operation (moving, multiple sensors active). Collect performance metrics (latency, packet loss, BER).

Week 11/24 (Thanksgiving):

- Will: Continue work on documentation. Construct test environment.
- Jacob: Off week/ Documentation for final paper where needed.
- Zach: Optimize algorithm parameters under realistic robot operation and write JSON files for test environment.
- Karthik: Debug issues (e.g., dropped packets, incorrect GPIO timing). Add error handling (re-sending commands, timeout detection). Work on Final Paper.

Week 12/01 (EXPO):

- Will: Test motion in the final test environment. Address potential issues.
- Jacob: Final tests/verifications. Runtime test for over an hour. Documentation for final paper.
- Zach: Finalize GUI and manual control.
- Karthik: Finalize firmware for live demo. Ensure stable motor control + sensor communication. Work on Final Paper.

Final Presentation – 12/3:

- Present all module results

EXPO – 12/4:

- Live demo of all modules in full robot pipeline.

Week 12/08:

- Will: Documentation for final paper.
- Jacob: Documentation for final paper.
- Zach: Documentation for final paper.
- Karthik: Documentation for final paper.

Final Document Due – 12/12:

- Controls section completed and polished.
- Firmware section completed and polished.
- Power section completed and polished.
- Software section completed and polished.

8.2 Project Budget

Item Name	Vendor	Unit Price	Item Count	Total Price
ESP-32 SMD Chip	DigiKey	$5.06	3	$15.18
Adafruit Micro-Lipo Charger for LiPoly Batt with USB Type C Jack	Adafruit	$5.95	1	$5.95
Micro Lipo 2000	DigiKey	$11.59	1	$11.59
N20 Gear Motor	Amazon	$13.99	1	$13.99
DRV8833 Motor Drivers	Amazon	$7.89	1	$7.89
ArduCam Mini Camera Module	Amazon	$25.99	2	$51.98
Flex Sensor	Amazon	$8.81	1	$8.81
AA Battery Housing	DigiKey	$1.30	1	$1.30
4.7 uF 6.3 V Capacitor	DigiKey	$0.17	1	$0.17
100 nF 10 V Capacitor	DigiKey	$0.49	1	$0.49

Table 3: Initial Project Budget BOM

The project budget breakdown is indicated by the table above. The table indicates the item name, vendor name, unit price, item count, and total price. This initial order mainly focuses on the components needed to implement the minimum working model for each of the modules. For the firmware, this would be a simple test to collect and transmit sensor data. Obtaining these components allows for early testing and any deviation that might result in our initial plan. Some of the things that these components will be used for are custom PCB for the second ESP32 being used, camera module, sensors, motors, and motor drivers as well. By making these initial purchases, we ensure that the project progresses according to the proposed schedule above, while still staying within the $200 budget we have.

8.3 Minimum Standard for Project Completion

The minimum standard for this project completion will be to demonstrate a robot that will be able to navigate through an obstacle room and change its shape depending on the terrain it is currently attempting to navigate through. While the 3D reconstruction feature may not be functional in this standard, the robot must still demonstrate the sensor data collection, communication, control logic, efficient power distribution, and folding commands properly guiding the robot.

8.4 Final Demonstration

For the final demonstration, the robot will navigate a laser-cut obstacle room, autonomously changing its shape as needed, while simultaneously performing 3D reconstruction of the environment in the interface. The robot will utilize all of the sensors and cameras embedded in it in order to accomplish this in real-time. The system will showcase the full integration of hinge/folding mechanism, efficient power distribution, sensor data and communication, and real-time visualization of the room reconstruction, demonstrating both the folding capabilities of the robot and its computer vision functionality.

The test cases for this final demonstration will be dependent on the shape of the obstacle room we make. The room will have different obstacles that would require the robot to change its shape to pass thorough tight spaces. Each of the obstacles would test the robot's ability to understand the sensor data properly and react accordingly. Furthermore, the 3D reconstruction output will be compared against the actual map of the room to see how accurate the robot was. Success will be measured on how close to the actual room shape it was.

References

[1] "Our Lady of the Angels School Fire Report," olafire.com. [Online]. Available: https://olafire.com/NFPareport.asp

[2] L. Noriega, "Rescue Robots Could Save Lives in Dangerous Disasters," Freethink, Oct. 27, 2020. [Online]. Available: https://www.freethink.com/hard-tech/rescue-robots

[3] K. Yang et al., "Origami Robots: A review of current technologies, applications, and future perspectives," arXiv preprint arXiv:2301.04743, Jan. 2023. [Online]. Available: http://arxiv.org/pdf/2301.04743

[4] A. Dzarrouk, R. Fearing, and M. Spenko, "Sprawlita: A sprawl-tuned, low cost hexapedal robot," Proc. IEEE Int. Conf. on Robotics and Automation (ICRA), Karlsruhe, Germany, May 2013. [Online]. Available: https://people.eecs.berkeley.edu/~ronf/PAPERS/dzarrouk-sprawl-icra13.pdf

[5] “NiMH vs LiPo Batteries: What Is The Difference And How to Choose,” EBL Official Blog, Jun. 16, 2025. [Online]. Available: https://www.eblofficial.com/blogs/comparison-hub/nimh-vs-lipo-battery

[6] X. Liu, S. Zhang, Y. Li, and C. Wang, “Review on flexible printed circuit board technology,” Frontiers of Mechanical Engineering, vol. 15, no. 4, pp. 748–766, Dec. 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2095495620307075

[7] EBL Official, “EBL Lithium Vs Ni-MH Rechargeable AA Batteries Review,” YouTube. [Online]. Available: https://www.youtube.com/watch?v=8npqPz5fvnI

[8] INJOINIC, IP2312 Battery Charger IC Datasheet (Translated), 2021. [Online]. Available: https://make.net.za/wp-content/datasheets/INJOINIC%20IP2312%20Translated.pdf

[9] “What’s PCM of LiPo Battery,” LiPol Battery, 2024. [Online]. Available: https://www.lipolbattery.com/What%27s-PCM-of-LiPo-Battery.html

[10] Texas Instruments, TPS2116: 1.6-V to 5.5-V, 2.5-A Low IQ Power Mux with Manual and Priority Switchover, Data Sheet, Rev. A, May 2021. [Online]. Available: https://www.ti.com/lit/ds/symlink/tps2116.pdf

[11] Texas Instruments, “Powering a High-Performance Processor with a Power Mux,” Application Report SLYT842, 2020. [Online]. Available: https://www.ti.com/lit/an/slyt842/slyt842.pdf

[12] Texas Instruments, “Simple Power MUX with Seamless Transition,” Application Report SLVA959B, Nov. 2017. [Online]. Available: https://www.ti.com/lit/an/slva959b/slva959b.pdf

[13] R. Peruzzi, “What Are the Basic Guidelines for Layout Design of Mixed-Signal PCBs,” Analog Dialogue, vol. 51, no. 2, 2017. [Online]. Available: https://www.analog.com/en/resources/analog-dialogue/articles/what-are-the-basic-guidelines-for-layout-design-of-mixed-signal-pcbs.html

[14] “Types of Flexible Circuits,” Epec Engineered Technologies, 2021. [Online]. Available: https://www.epectec.com/flex/types/

[15] Texas Instruments, “Battery Cell Balancing – What to Balance and How,” TI Training Seminar Notes, 2017. [Online]. Available: https://www.ti.com/download/trng/docs/seminar/Topic%202%20-%20Battery%20Cell%20Balancing%20-%20What%20to%20Balance%20and%20How.pdf

[16] Texas Instruments, BQ40Z80 Multi-Cell Li-Ion Battery Fuel Gauge, Data Sheet, Rev. A, 2019. [Online]. Available: https://www.ti.com/lit/ds/symlink/bq40z80.pdf

[17] S. Knoth, “Supply Clean Power with Ultralow Noise LDO Regulators,” Analog Devices Technical Article, Sep. 2018. [Online]. Available: https://www.analog.com/en/resources/technical-articles/supply-clean-power-with-ultralow-noise-ldo-regulators.html

[18] Texas Instruments, DRV8833 Dual H-Bridge Motor Driver, Data Sheet, Rev. B, 2015. [Online]. Available: https://www.ti.com/lit/ds/symlink/drv8833.pdf

[19] Salmony, P. (n.d.). *PID Controller Implementation in Software - Phil’s Lab #6*. YouTube. [Online]. Available: https://www.youtube.com/watch?v=zOByx3Izf5U

[20] Developer, W, Wiring flashing / programming ESP-32 / esp32s with USB – TTL / UART and integration with Arduino IDE. 14core. [Online]. https://www.14core.com/wiring-and-flashing-programming-esp-32-esp32s-with-usb-ttl-uart/amp/

[21] EspressIf. (n.d.). ESP-NOW Wireless Communication Protocol. [Online]. Available: https://www.espressif.com/en/solutions/low-power-solutions/esp-now

[22] Random Nerd Tutorials. (n.d.). ESP32: ESP-NOW Encrypted Messages. [Online]. Available: https://randomnerdtutorials.com/esp32-esp-now-encrypted-messages/